GUIA DO ALUNO / PORTUGUÊS

DryRun Security

Aprenda praticando, passo a passo

Da preparação à remediação com IA

Sem experiência prévia em Git ou terminal

---

01 **PREPARAR**

Ferramentas, contas e repositório seguro

02 **EXECUTAR**

25 módulos com 167 passos guiados

03 **CONFERIR**

Resultados esperados e evidências

04 **RECUPERAR**

Diagnóstico, retomada e limpeza

GITHUB / DRYRUN API / CODEX & MCP / JIRA

Edição 2 • 23/10/2026 • Treinamento independente

Sumário

Como estudar este treinamento 3
Como ler os blocos 4
Preparação inicial - faça uma vez 5
Retomar o curso em outro dia 7
Preparação adicional - faça quando chegar à integração 8
01. Documentation 9
02. Quick Start 11
03. PR Scanning 14
04. SCA - Dependencies 17
05. Repository Scanning with DeepScan 19
06. Secrets Scanning 21
07. IaC Scanning 24
08. Language and Framework Support 27
09. Vulnerability Coverage Matrix 29
10. Reporting 31
11. Risk Register 33
12. Code Security Intelligence 36
13. PR Scanning Configuration 38
14. Custom Code Policies 41
15. Repository Context 44
16. Finding Tuning with Feedback 47
17. PR Blocking 49
18. Permissions 52
19. MCP 54
20. DryRun API 56
21. Webhooks 59
22. Jira Integration 63
23. API Access Keys 66
24. Remediation 68
25. DryRun Skill 71
Exercício final - demonstre o ciclo completo 73
Diagnóstico rápido e retomada 74
Apêndice A - código completo do kit 75
Apêndice B - fontes e limites 80

Como estudar este treinamento

Esta edição foi reescrita para quem ainda não tem experiência com Git, terminal ou integrações de segurança. Os 25 módulos mantêm a explicação técnica e o caso de uso, mas os laboratórios agora usam passos numerados, local de execução, resultado esperado e orientação de recuperação. A versão em inglês tem os mesmos módulos, objetivos e comandos.

Leia primeiro a preparação. Faça os módulos 1 a 18 na sequência; antes dos módulos 19 e 20, conclua a preparação de Codex e chave DryRun. O módulo 21 exige um receptor local e, para entrega externa, um túnel temporário. O módulo 22 exige Jira Cloud. Quando um acesso não estiver disponível, registre a etapa como não executada; não trate isso como sucesso nem como finding inexistente.

A estimativa didática é de 30 a 40 horas, incluindo leitura, prática e diagnóstico. Faça sessões de 2 a 3 horas e salve seu caderno. Scans e estatísticas podem levar tempos diferentes conforme fila, repo e conta. As referências principais continuam sendo os PDFs fornecidos em 20/10/2026; esta revisão pedagógica é de 23/10/2026.

Não há flashcards. Este material é independente do fornecedor e não é uma tradução literal da documentação. O item “DryRun” do programa original corresponde ao módulo DryRun API. Os arquivos de referência são fontes técnicas, não instruções para alterar suas contas durante a elaboração deste documento.

O vocabulário antes da prática

Terminal é a janela em que você executa comandos. Shell é o programa que os interpreta; este curso usa Bash. Pasta atual é o local onde os comandos operam: pwd mostra esse caminho e ls lista arquivos. $HOME é sua pasta de usuário. Não substitua o valor dessa variável.

Repositório é uma pasta com histórico Git. Branch é uma linha de trabalho; main é a base segura deste curso. Commit salva uma versão local. Push envia commits ao GitHub. Pull request (PR) é o pedido de revisão de uma branch antes de integrá-la à base. Merge integra o trabalho; fechar uma PR sem merge não o integra.

Scan é uma execução de análise. Finding é um achado que precisa de investigação. SHA identifica um commit. Check é o resultado associado a um commit no GitHub. Um check de falha pode ser a detecção esperada no laboratório; não significa automaticamente problema de instalação.

API é uma interface programática. Token/chave é uma credencial: não a coloque em arquivos versionados. JSON é um formato de dados; jq ajuda a lê-lo. Webhook é uma requisição enviada quando um evento ocorre. MCP conecta um cliente de IA a ferramentas/contexto; skill fornece instruções de trabalho ao agente.

Como ler os blocos

Execute um bloco de cada vez e confira o resultado antes do seguinte. Não copie a marcação ``` nem o nome bash. Linhas começadas por # são comentários. Uma barra invertida no fim da linha continua o comando na linha seguinte: copie o bloco completo.

Blocos com PROMPT, RULE, BACKGROUND ou COMMENT FORMAT são texto para o chat/interface indicado, nunca comandos. Em blocos cat <<'TXT', copie também a linha final TXT, sem espaços. Esse recurso cria um arquivo. Se aparecer apenas > esperando entrada, talvez falte o delimitador; Ctrl+C cancela para tentar novamente.

Os comandos read pedem um valor depois de Enter. Digite sua informação e pressione Enter. read -s não mostra caracteres enquanto você cola uma credencial: isso é esperado. Valores como OWNER/REPO em explicações são exemplos que precisam ser substituídos. Nunca digite IDs de exemplo como se fossem os seus.

“Esperado” é o ponto de conferência. “Se não acontecer” explica a recuperação. Pare quando uma etapa obrigatória falhar; executar os próximos passos às cegas geralmente cria um erro secundário. As funções lab_* e dryget são auxiliares autorais do curso, não uma CLI oficial DryRun. Seu código completo está no apêndice.

Preparação inicial - faça uma vez

1. Abra o terminal e escolha seu sistema

No macOS, pressione Command+Espaço, digite Terminal e abra o aplicativo. No Linux, abra o aplicativo Terminal. No Windows, utilize uma sessão Ubuntu em WSL2 já instalada pela sua equipe; os comandos não são para PowerShell. O caminho principal abaixo usa Homebrew no macOS ou Linux/WSL compatível. Se instalações forem gerenciadas pela empresa, peça essas ferramentas à equipe de TI em vez de contornar a política.

Digite o comando abaixo e pressione Enter. Você passa a usar Bash nesta janela, mesmo que o padrão do macOS seja zsh.

```
bash
```

2. Instale as ferramentas básicas

Primeiro confira se Homebrew já existe. Se aparecer um caminho, pule a instalação e vá ao comando brew install.

```
command -v brew
```

Se não aparecer nada, use o instalador oficial abaixo. Leia o que ele informa e siga suas perguntas. Pode solicitar senha do sistema e ferramentas de desenvolvimento. Se parar com erro, resolva a instalação antes de continuar. Referência: https://brew.sh

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

O instalador mostra “Next steps”. Execute as instruções que ele fornece. Para habilitar o caminho nesta sessão, uma das localizações abaixo deverá existir; o bloco seleciona a correspondente ao sistema.

```
if [ -x /opt/homebrew/bin/brew ]; then
  eval "$(/opt/homebrew/bin/brew shellenv)"
elif [ -x /usr/local/bin/brew ]; then
  eval "$(/usr/local/bin/brew shellenv)"
elif [ -x /home/linuxbrew/.linuxbrew/bin/brew ]; then
  eval "$(/home/linuxbrew/.linuxbrew/bin/brew shellenv)"
fi
brew install git gh jq python node
```

Se Homebrew pedir pré-requisitos no Linux/WSL Ubuntu, instale-os e retome o instalador. Não execute este bloco no macOS.

```
sudo apt-get update
sudo apt-get install -y build-essential procps curl file git openssh-client
```

Confira as ferramentas. Cada comando deve imprimir versão ou caminho. command not found exige corrigir instalação/PATH. curl deve aceitar --fail-with-body; se o curl do sistema for antigo, instale uma versão recente pelo gerenciador e confirme que ela é a versão no PATH.

```
git --version
gh --version
jq --version
python3 --version
node --version
npm --version
curl --version
command -v ssh-keygen
```

3. Entre no GitHub

Execute o login. No assistente, escolha GitHub.com, HTTPS e login pelo navegador. Copie o código temporário que a ferramenta apresentar, abra a página indicada e autorize sua própria conta. Os rótulos podem mudar, mas o objetivo é autenticar a CLI com a conta que tem acesso ao repo de exercícios.

```
gh auth login
gh auth setup-git
gh auth status
```

Esperado: sua conta aparece autenticada em github.com. Se for outra conta, corrija a seleção antes de criar repo. setup-git configura a autenticação do Git para usar a CLI; ele não cria repositórios.

4. Baixe o material e confira os arquivos

Este clone guarda os materiais, não é o alvo dos scans. O módulo 2 cria outro repo para exercícios. Não crie branches vulneráveis dentro de material.

```
mkdir -p "$HOME/dryrun-student"
git clone https://github.com/lca-appsec/DryRun-training.git \
  "$HOME/dryrun-student/material"
ls "$HOME/dryrun-student/material/laboratorios"
mkdir -p "$HOME/dryrun-student/notes"
```

Você deve ver app.py, test_app.py, lab.sh e save_session.py. Se a pasta já existir, não repita clone: confira o repo e atualize os materiais somente se não houver mudanças locais.

```
git -C "$HOME/dryrun-student/material" status --short
git -C "$HOME/dryrun-student/material" pull --ff-only
```

Alternativa offline: extraia o kit ZIP em $HOME/dryrun-student/material/laboratorios. Nesse caso essa pasta de material não será um clone Git, então não execute pull nela. A criação do repo de exercícios continuará exigindo rede.

5. Confira os acessos

Abra https://app.dryrun.security e confirme sua organização. A instalação da App para o repo novo será feita no módulo 2. Para configurações, políticas, DeepScan e chaves, confirme o papel administrativo necessário. Jira e Codex exigem acessos próprios; sua conta GitHub não os habilita automaticamente.

Retomar o curso em outro dia

Depois do módulo 2, o arquivo session.sh guarda caminhos e OWNER/REPO, sem chaves. Em qualquer terminal novo, rode os comandos abaixo. Não recrie o repo nem copie novamente app.py por cima do trabalho.

```
bash
source "$HOME/dryrun-student/session.sh"
pwd
git branch --show-current
git status --short
```

Esperado: pwd termina em dryrun-student/repo. Se você parou em lab/03, por exemplo, retome o passo desse módulo. lab_branch cria uma branch nova e não deve ser repetido se ela já existir. Para inspecionar uma PR existente, use gh pr view --web na branch correspondente.

Se um comando lab_* não existir, carregue session.sh. Se session.sh não existir mas o repo já estiver criado, entre na pasta repo, confirme o remote e execute save_session.py com o OWNER/REPO real. O comando abaixo pede esse nome em vez de presumir um valor.

```
cd "$HOME/dryrun-student/repo"
git remote -v
read -rp 'Existing exercise OWNER/REPO: ' GH_REPO
python3 "$HOME/dryrun-student/material/laboratorios/save_session.py" "$GH_REPO"
source "$HOME/dryrun-student/session.sh"
```

Para src refspec main does not match any: rode git status, git branch --show-current e git log -1. Sem commit inicial, conclua o módulo 2. Se a branch tiver outro nome, entenda o histórico antes de renomear. Não use force push nem reset --hard para resolver a preparação. Não copie o texto HTML   para comandos.

Preparação adicional - faça quando chegar à integração

Chave DryRun - antes dos módulos 19 e 20

No navegador DryRun, abra Settings > Access Keys, escolha criar uma chave e dê um nome como training-read. Selecione pelo menos um escopo; escolha os de leitura necessários para os dados do exercício. A lista de escopos deve ser conferida na sua conta, não inferida do nome de um endpoint. Salve, copie o segredo exibido uma vez e guarde em um gerenciador seguro.

No terminal em que você executará a API ou abrirá Codex, cole o token quando solicitado. A tela não mostra o que você digita. Depois da leitura, Enter conclui.

```
source "$HOME/dryrun-student/session.sh"
read -rsp 'DryRun API key: ' DRYRUN_API_KEY; echo
export DRYRUN_API_KEY
dryget accounts
```

Esperado: resposta JSON com as contas autorizadas. Se 401, confira o token; se 403, confira escopo/papel. Não coloque o token no session.sh, README ou Git. Ao abrir outro terminal, leia/exporte a chave novamente. O módulo 23 explica rotação e revogação.

Codex - antes dos módulos 19, 24 e 25

Instale e autentique a CLI no terminal. Siga a página de login aberta e volte ao terminal para confirmar o estado. Referências oficiais: https://learn.chatgpt.com/docs/cli/reference e https://learn.chatgpt.com/docs/codex/cli

```
npm install -g @openai/codex
codex --version
codex login
codex login status
```

Se npm informar permissão negada, use uma instalação Node gerenciada para seu usuário ou solicite suporte à TI; não acrescente sudo às cegas. O login precisa de uma conta autorizada a usar Codex. O módulo 19 configura MCP separadamente.

cloudflared - antes da entrega externa do módulo 21

Instale pelo mesmo Homebrew da preparação. Esse cliente cria uma URL HTTPS temporária para um receptor local; não é necessário abrir um serviço de produção.

```
brew install cloudflared
cloudflared --version
```

Jira Cloud: obtenha site, projeto de teste, permissão de criação e token pessoal antes do módulo 22. Os comandos de consulta, payload e criação ficam no próprio módulo. Não use um projeto de produção como caixa de testes.

GETTING STARTED

01. Documentation

Mecanismo e decisões técnicas

DryRun usa análise contextual de código para relacionar uma alteração às funções, fluxos e controles de segurança que a cercam. A unidade de trabalho muda conforme o recurso: PR Scanning analisa mudanças propostas; DeepScan explora um repositório em uma referência; SCA examina dependências; o Risk Register consolida achados para tratamento. Confundir essas unidades leva a interpretar um scan de PR limpo como avaliação completa de uma aplicação.

O fluxo operacional começa no SCM, onde o evento da PR inicia a análise. Os resultados alimentam o comentário e os checks, o painel e as integrações habilitadas. Um status check reporta um resultado; uma regra de proteção do GitHub transforma esse resultado em impedimento de merge. O painel permite investigação e triagem; as APIs e o MCP expõem contextos diferentes para automação e perguntas.

LLMs participam da interpretação contextual e de recursos como Code Security Intelligence. A configuração, o código e os documentos de contexto ajudam a formular essa interpretação. MCP é o protocolo de integração entre um cliente de IA e ferramentas expostas pelo servidor; não é um scanner nem uma substituição da proteção de branch. Uma resposta do assistente precisa ser rastreável às evidências retornadas pelas ferramentas.

Caso de uso

Uma equipe deseja saber se um patch em autorização cria um IDOR. O fluxo correto é examinar o diff e a rota no PR Scanning, entender o modelo de autorização com Repository Context, conferir alcance e variantes com DeepScan e registrar a decisão de tratamento. Um resumo do LLM sem caminhos, linhas ou finding IDs não basta para aprovar a mudança.

Antes de começar

Preparação inicial concluída; navegador com acesso ao GitHub e DryRun. Ainda não é necessário ter o repo de exercícios.

Laboratório guiado 01

Passo 1

Terminal - confira qual conta GitHub está autenticada. O comando consulta sua identidade e não altera repositórios.

```
gh auth status
gh api user --jq .login
```

Esperado: Anote o login retornado. Ele será usado no módulo 2.

Se não acontecer: Se aparecer erro de autenticação, repita gh auth login conforme a preparação.

Passo 2

Navegador - abra https://app.dryrun.security, entre com sua conta e selecione a organização correta. Localize Repository Scanning/DeepScan, Risk Register e Settings no menu; os rótulos podem variar.

Esperado: Você consegue identificar onde ver scans, onde tratar findings e onde configurar o produto.

Se não acontecer: Se a organização não aparecer, solicite acesso ao administrador; não use outra organização por conveniência.

Passo 3

Terminal - crie um caderno local. Substitua as linhas em branco em um editor de texto simples. Esse arquivo não será enviado ao GitHub.

```
mkdir -p "$HOME/dryrun-student/notes"
cat > "$HOME/dryrun-student/notes/01.txt" <<'TXT'
GitHub login:
DryRun organization:
My role:
PR scan = changed code
DeepScan = repository analysis
TXT
```

Esperado: Salve a identidade e o papel que aparecem na plataforma.

Se não acontecer: Se não souber o papel, anote “não confirmado” e confirme antes de mudar configurações.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Documentation, páginas 1-2; Quick Start, páginas 1-7.

GETTING STARTED

02. Quick Start

Mecanismo e decisões técnicas

A GitHub App precisa ter acesso ao repositório antes de receber eventos relevantes. Permitir uma organização não implica necessariamente selecionar todos os repositórios: instalações limitadas a repositórios específicos precisam incluir o novo laboratório. O quick start deve terminar com uma PR rastreável, não apenas com o aplicativo instalado.

Uma base inicial com código seguro permite comparar o comportamento antes e depois de cada alteração. A main conterá a aplicação mínima e dois testes: consulta normal e payload tratado como valor literal. Cada laboratório começa de main, abre uma branch própria e fecha a PR sem merge quando houver código vulnerável. Isso evita que o exercício seguinte herde uma falha anterior.

Caso de uso

Você quer demonstrar DryRun em uma conta que já tem vários projetos. Um repositório novo permite aplicar configurações e checks sem alterar regras de produção. Ele também evita ambiguidade com o remote SSH github-veracode e com a branch local main inexistente do erro de push observado anteriormente.

Antes de começar

Módulo 1 e preparação concluídos. Escolha uma conta/organização onde você possa criar um repo privado e autorizar a App.

Laboratório guiado 02

Passo 1

Terminal - execute o comando e digite sua conta/organização quando solicitado. Escolha um nome de repo novo, por exemplo dryrun-student-luis. Não use DryRun-training, que guarda o material do curso.

```
read -rp 'GitHub owner: ' GH_OWNER
read -rp 'New exercise repository name: ' LAB_NAME
export GH_REPO="$GH_OWNER/$LAB_NAME"
mkdir -p "$HOME/dryrun-student/repo"
cd "$HOME/dryrun-student/repo"
git init -b main
read -rp 'Your commit name: ' COMMIT_NAME
read -rp 'Your commit email: ' COMMIT_EMAIL
git config --local user.name "$COMMIT_NAME"
git config --local user.email "$COMMIT_EMAIL"
```

Esperado: git init cria a pasta .git e indica main. Nome/email identificam seus commits; não são senhas.

Se não acontecer: Se a pasta já contiver um repo do curso, não reinicialize nem sobrescreva arquivos; use a seção Retomar.

Passo 2

Terminal - copie apenas a aplicação e seus testes do kit. Execute o teste antes de criar a primeira versão.

```
export KIT="$HOME/dryrun-student/material/laboratorios"
cp "$KIT/app.py" "$KIT/test_app.py" .
printf '.evidence/\n__pycache__/\n.env\n.venv/\n' > .gitignore
printf '# DryRun student repository\n' > README.md
python3 -m unittest -v
```

Esperado: Você deve ver dois testes e OK. Não há servidor para instalar ou banco externo.

Se não acontecer: Se houver ModuleNotFoundError para app, execute pwd e ls; app.py e test_app.py devem estar nesta pasta.

Passo 3

Terminal - salve o primeiro commit e envie para um repo privado novo. Isso publica a base segura, não a aplicação na Internet.

```
git add app.py test_app.py .gitignore README.md
git commit -m 'Safe student baseline'
gh repo create "$GH_REPO" --private --source . --remote origin --push
python3 "$KIT/save_session.py" "$GH_REPO"
source "$HOME/dryrun-student/session.sh"
```

Esperado: O GitHub retorna a URL do repo. session.sh guarda caminhos e o nome do repo, sem tokens.

Se não acontecer: Se o nome já existir, não force o push. Confirme se é o seu repo de exercício e retome-o, ou escolha outro nome.

Passo 4

Navegador - na instalação da GitHub App DryRun, use Configure e inclua o novo repo em Repository access. No DryRun, confirme que esse repo aparece. Se não for administrador, peça ao administrador para fazer esse passo.

```
gh repo view "$GH_REPO" --web
```

Esperado: O mesmo nome OWNER/REPO aparece no GitHub e no DryRun.

Se não acontecer: Não prossiga com o scan se a instalação ainda não incluir o repo.

Passo 5

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/02
```

Esperado: A última mensagem deve indicar a branch lab/02. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 6

Terminal - faça uma alteração de documentação e salve um commit.

```
printf '\nFirst onboarding PR.\n' >> README.md
git add README.md
git commit -m 'Test DryRun onboarding'
```

Esperado: O commit contém apenas README.md.

Se não acontecer: Use git diff --stat e git status para conferir arquivos inesperados antes de publicar.

Passo 7

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 02 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 8

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 9

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Quick Start, páginas 1-7.

SCANNING

03. PR Scanning

Mecanismo e decisões técnicas

O scan de PR avalia o diff e consulta o contexto necessário para entender seu efeito. Abertura e atualizações da PR geram novas análises. O código legado fora da alteração não deve ser tratado como integralmente revisado. Drafts podem ser analisados, mas os comentários aguardam a PR ficar pronta para revisão.

Por padrão, uma análise sem findings não produz comentário. Se os findings forem corrigidos em um commit seguinte, o comentário pode ser removido. A opção Show comment for no findings altera isso para manter uma confirmação explícita. Checks e registro de scan são evidência mais confiável do que presença ou ausência de comentário.

O laboratório usa uma SQL injection em SQLite: concatenar a entrada ao SQL muda a estrutura da consulta. Parametrizar preserva a entrada como dado. O teste local demonstra essa diferença mesmo que o scanner não produza o finding esperado. A análise automatizada e a prova funcional são evidências complementares.

Caso de uso

Um desenvolvedor substitui uma consulta parametrizada por concatenação para “simplificar” um filtro. O fluxo de dados vem da query string HTTP, chega à função search e termina em execute. Esse encadeamento permite avaliar controlabilidade da entrada e impacto da operação.

Antes de começar

Módulo 2 concluído; main segura; repo visível no DryRun.

Laboratório guiado 03

Passo 1

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/03
```

Esperado: A última mensagem deve indicar a branch lab/03. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 2

Terminal - introduza a falha somente nesta branch e rode os testes. unsafe significa “inseguro”.

```
python3 "$KIT/change.py" unsafe
python3 -m unittest -v
git diff -- app.py
```

Esperado: O teste test_injection_is_literal deve falhar; o teste de busca normal passa. Aqui a falha é esperada: ela prova o problema.

Se não acontecer: Se ambos passarem, pare: confira se a alteração foi aplicada e se você está na pasta repo correta.

Passo 3

Terminal - salve a versão vulnerável para enviá-la à revisão.

```
git add app.py
git commit -m "Lab: unsafe SQL query"
```

Esperado: O diff do commit mostra concatenação SQL.

Se não acontecer: Se aparecer nothing to commit, confira git log -1; talvez o passo já tenha sido executado.

Passo 4

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 03 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 5

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 6

DryRun - abra o finding, se existir. No caderno, anote ID, arquivo, linha, entrada controlada, operação SQL e severidade. Compare a explicação com app.py.

```
nl -ba app.py
```

Esperado: Você consegue explicar por que a concatenação permite alterar a consulta.

Se não acontecer: Um scan sem finding não invalida o teste local que demonstrou a falha. Registre ambos.

Passo 7

Terminal - corrija, teste e envie um novo commit à mesma PR.

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Fix SQL query using bound parameters'
git push
```

Esperado: Dois testes passam. A mesma PR ganha um novo SHA e um novo scan.

Se não acontecer: Não feche a PR antes de conferir o resultado do commit corrigido.

Passo 8

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 9

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: PR Scanning, páginas 1-5; PR Scanning Configuration, páginas 3-4.

SCANNING

04. SCA - Dependencies

Mecanismo e decisões técnicas

SCA identifica componentes e vulnerabilidades conhecidas a partir dos manifests e lockfiles reconhecidos. A versão resolvida é importante: um intervalo no manifesto não comprova a versão efetivamente instalada. O lockfile ajuda a reconstruir a árvore de dependências e sua relação com o projeto.

A documentação descreve SCA dentro de DeepScan e scans em segundo plano. A frequência destes depende da atividade de PRs nos últimos 30 dias; repositórios com menos de três PRs não entram nesse agendamento. Por isso, não espere que um laboratório novo receba imediatamente o scan de fundo. Use DeepScan para executar o percurso de forma controlada.

Em Gradle, build.gradle isolado não basta para resolver a lista de componentes: utilize um lockfile suportado e commitado. A documentação também descreve verification-metadata.xml na raiz para o caso suportado. SBOM é inventário de componentes; ele não é uma lista equivalente de vulnerabilidades. Ausência de CVE associada não torna automaticamente um componente confiável.

Caso de uso

A equipe precisa comprovar quais versões foram avaliadas antes de atualizar uma biblioteca. Exporta o SBOM, identifica dependências diretas e transitivas, decide a atualização e executa testes de compatibilidade. A decisão não deve usar apenas a contagem de CVEs.

Antes de começar

Módulo 2; acesso para iniciar DeepScan. Não é necessário instalar requests.

Laboratório guiado 04

Passo 1

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/04
```

Esperado: A última mensagem deve indicar a branch lab/04. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 2

Terminal - crie um manifesto com uma versão antiga para análise. O comando escreve um arquivo; não instala a biblioteca.

```
printf 'requests==2.19.1\n' > requirements.txt
git add requirements.txt
git commit -m 'Lab: dependency manifest'
```

Esperado: requirements.txt contém exatamente requests==2.19.1.

Se não acontecer: Não execute pip install neste exercício. O objetivo é analisar o manifesto commitado.

Passo 3

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 04 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 4

DryRun - abra Repository Scanning/DeepScan, selecione seu repo e escolha a branch lab/04 no seletor de referência. Inicie o scan. Anote a referência antes de confirmar.

```
git rev-parse HEAD
```

Esperado: A execução mostra a branch do laboratório. Aguarde o estado final antes de abrir dependências.

Se não acontecer: Se você só puder selecionar main, pare e confirme o fluxo de branch com o administrador; main não contém esse manifesto.

Passo 5

DryRun - abra os resultados SCA/dependências da execução, procure requests e confira a versão. Exporte o SBOM pelo controle disponível e salve em .evidence pelo navegador.

Esperado: Registre componente, versão e finding/advisory se houver. Não há promessa de um CVE específico.

Se não acontecer: Se o componente não aparecer, confira branch, arquivo commitado, formato suportado e conclusão da etapa SCA.

Passo 6

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: SCA, páginas 1-6; DryRun API, páginas 1-6.

SCANNING

05. Repository Scanning with DeepScan

Mecanismo e decisões técnicas

DeepScan é a avaliação de repositório inteiro, inclusive código não alterado recentemente. A sequência documentada inclui perfil do projeto, mapeamento de rotas e autenticação, planejamento por domínios de segurança, investigação, consolidação/calibração e relatório. Isso o torna adequado para baseline, aquisição de uma aplicação ou revisão de superfícies legadas.

O histórico Git fornece sinais para orientar investigação: frequência de mudanças, autores, recência, complexidade e arquivos que costumam mudar juntos. Esses sinais ajudam a priorizar atenção, mas não são prova de vulnerabilidade nem avaliação individual de desempenho de um desenvolvedor.

O scan pode ser iniciado para uma branch pela interface. Registre a referência e o commit analisados: um relatório antigo não valida o estado atual. Contexto específico configurado para DeepScan substitui o conteúdo de AGENTS.md para essa análise, enquanto, na ausência desse contexto, DeepScan pode descobrir arquivos AGENTS.md inclusive em subdiretórios.

Caso de uso

Uma API com poucos PRs recentes tem um controle de autorização vulnerável há meses. PR Scanning não deve ser usado como auditoria desse legado. DeepScan fornece uma avaliação mais ampla, que depois é convertida em remediações pontuais com PRs verificáveis.

Antes de começar

Módulo 2; acesso Admin ou permissão compatível para DeepScan.

Laboratório guiado 05

Passo 1

Terminal - volte à main e anote seu commit. Não precisamos criar uma PR para este exercício.

```
source "$HOME/dryrun-student/session.sh"
git status --short
git switch main
git rev-parse HEAD
```

Esperado: A árvore está limpa e você tem o SHA da main.

Se não acontecer: Se houver alterações, termine o laboratório anterior antes de trocar de branch.

Passo 2

DryRun - abra DeepScan, selecione o repo e main, revise o alvo e inicie uma execução. Anote horário e identificador exibido.

Esperado: Uma execução aparece na fila ou em processamento. Isso ainda não é um resultado final.

Se não acontecer: Se o botão não estiver disponível, confira papel e recursos da conta; não tente repetir a ação por API para contornar permissão.

Passo 3

DryRun - acompanhe as fases até concluir. Abra o relatório e localize linguagens, rotas/superfícies e findings quando disponibilizados. Compare com app.py.

```
nl -ba app.py
```

Esperado: O relatório corresponde ao alvo selecionado. Um baseline pequeno e seguro pode ter zero findings.

Se não acontecer: Se houver erro, salve a mensagem e o ID. Não inicie vários scans idênticos para tentar acelerar a fila.

Passo 4

Caderno - salve repo, branch, commit, ID, estado final e uma observação sobre cobertura. Mantenha a main; não há PR a fechar.

```
git status --short
```

Esperado: Você consegue ligar um relatório a uma versão exata do código.

Se não acontecer: Se o commit do relatório não estiver disponível, registre essa limitação em vez de afirmar igualdade de SHA.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Repository Scanning with DeepScan, páginas 1-5.

SCANNING

06. Secrets Scanning

Mecanismo e decisões técnicas

O scanner de segredos procura credenciais no código e usa contexto para diferenciar material suspeito de exemplos e valores inofensivos. A documentação menciona tentativas de validação de atividade quando aplicável. Uma detecção não substitui a decisão operacional sobre revogação; um segredo real já commitado deve ser tratado como potencialmente exposto mesmo após remoção do arquivo.

Uma credencial fictícia pode não ser reportada por ter formato inválido ou contexto de teste. Para observar comportamento sem expor uma conta, o laboratório gera uma chave privada descartável que nunca será instalada, confiada ou usada para autenticação. O finding continua não sendo garantido: o exercício também avalia como documentar uma não detecção.

Caso de uso

Uma chave real é enviada por engano em uma PR. A ordem operacional é revogar/rotacionar no provedor, atualizar consumidores, confirmar funcionamento e só então tratar arquivos e histórico conforme o processo da equipe. Apagar a linha do diff não invalida cópias existentes da credencial.

Antes de começar

Módulo 2; ssh-keygen instalado pelo sistema. Nunca use uma chave pessoal.

Laboratório guiado 06

Passo 1

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/06
```

Esperado: A última mensagem deve indicar a branch lab/06. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 2

Terminal - gere uma chave descartável sem senha. Este material é exclusivo do exercício; não cadastre a chave pública no GitHub nem em servidores.

```
mkdir -p fixtures
ssh-keygen -t rsa -b 2048 -N '' \
  -C 'student-never-trusted' -f fixtures/lab_only_key
git add fixtures/lab_only_key fixtures/lab_only_key.pub
git commit -m 'Lab: disposable untrusted key'
```

Esperado: São criados dois arquivos: privado e .pub público.

Se não acontecer: Se pedir para sobrescrever arquivo, responda n e confira se este lab já foi feito.

Passo 3

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 06 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 4

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 5

DryRun - se houver finding, confira o arquivo apontado e a classificação. No caderno, registre o ID e a conclusão, sem copiar o conteúdo integral da chave.

Esperado: Você distingue material privado de público e sabe que a chave nunca foi confiada.

Se não acontecer: Se não houver finding, confira agente e configuração; não use um segredo real para forçar uma detecção.

Passo 6

Terminal - remova os dois arquivos no novo commit. Depois confira o scan atualizado e feche a PR.

```
git rm fixtures/lab_only_key fixtures/lab_only_key.pub
git commit -m 'Remove disposable key files'
git push
```

Esperado: Os arquivos saem da versão atual, mas continuam no histórico da branch.

Se não acontecer: Isso seria insuficiente para um segredo real; no exercício, a chave nunca foi usada nem confiada.

Passo 7

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 8

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Secrets Scanning, páginas 1-2.

SCANNING

07. IaC Scanning

Mecanismo e decisões técnicas

A documentação destaca Terraform (.tf) em PRs e riscos como permissões excessivas, exposição de rede, defaults inseguros e situações relacionadas a subdomain takeover. A análise é estática: não é necessário executar terraform apply nem criar infraestrutura para avaliar o código.

O contexto muda o significado de uma configuração: uma regra ingress para toda a Internet em uma porta administrativa merece investigação diferente de uma porta pública de serviço. Da mesma forma, uma política IAM ampla pode ser exigida por um fluxo específico, mas deve ter justificativa verificável e limites apropriados.

Caso de uso

Uma mudança libera SSH para 0.0.0.0/0. O review deve identificar a superfície criada, verificar a necessidade operacional e limitar a origem à rede de administração. O exercício compara uma abertura ampla com uma faixa de documentação, sem implantar recursos reais.

Antes de começar

Módulo 2. Não é necessário ter conta AWS nem executar terraform apply.

Laboratório guiado 07

Passo 1

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/07
```

Esperado: A última mensagem deve indicar a branch lab/07. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 2

Terminal - copie todo o bloco, incluindo a linha TF final. Ele cria o arquivo Terraform sem provisionar nada.

```
mkdir -p infra
cat > infra/security.tf <<'TF'
resource "aws_security_group" "lab" {
  name = "dryrun-student"
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
TF
git add infra/security.tf
git commit -m 'Lab: broad SSH ingress'
```

Esperado: O arquivo contém porta 22 e 0.0.0.0/0.

Se não acontecer: Se o terminal ficar esperando mais texto, confira se a linha TF foi copiada sem espaços. Ctrl+C cancela a entrada para tentar novamente.

Passo 3

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 07 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 4

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 5

Terminal - restrinja o exemplo a uma faixa reservada para documentação. Depois compare a mudança e envie.

```
python3 - <<'PYCODE'
from pathlib import Path
p = Path('infra/security.tf')
p.write_text(p.read_text().replace('0.0.0.0/0', '192.0.2.0/24'))
PYCODE
git diff
git add infra/security.tf
git commit -m 'Lab: narrow administrative source'
git push
```

Esperado: O diff muda apenas a faixa. Use uma origem real autorizada em um projeto real.

Se não acontecer: Não execute apply; a análise aqui é de código, não de infraestrutura implantada.

Passo 6

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 7

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: IaC Scanning, páginas 1-2.

SCANNING

08. Language and Framework Support

Mecanismo e decisões técnicas

A lista documentada inclui JavaScript/TypeScript, Python, Java, Go, Ruby, PHP, C#, Kotlin, Swift e Rust. O reconhecimento de linguagem e framework orienta a investigação sem exigir a mesma configuração de compilação de uma ferramenta baseada exclusivamente em build. Isso não implica cobertura idêntica de todos os frameworks, DSLs, geradores e bibliotecas de cada ecossistema.

Uma avaliação séria de cobertura verifica quatro elementos: linguagem reconhecida; entrada identificada; propagação compreendida; operação sensível contextualizada. Em um framework proprietário, Repository Context pode explicar wrappers de autenticação e rotas, mas não transforma automaticamente qualquer fluxo em análise garantida.

Caso de uso

Um monorepo possui serviço Python, frontend TypeScript e módulos Java. A equipe precisa validar caminhos representativos de cada stack antes de concluir que o produto cobre todo o conjunto. Um finding em Python não é evidência de entendimento do middleware Java.

Antes de começar

Módulos 2 e 5. Não há nova PR neste laboratório.

Laboratório guiado 08

Passo 1

Terminal - carregue sua sessão e liste os arquivos versionados. .py identifica arquivos Python.

```
source "$HOME/dryrun-student/session.sh"
git switch main
git ls-files
```

Esperado: Você vê app.py e test_app.py, além da documentação/configuração Git.

Se não acontecer: Se aparecerem arquivos dos labs anteriores, confirme que está em main e que não houve merge indevido.

Passo 2

Terminal - consulte a composição de linguagens no GitHub. Depois, no DryRun, abra o perfil do DeepScan do módulo 5.

```
gh api "repos/$GH_REPO/languages"
```

Esperado: Compare o reconhecimento de Python entre as duas fontes, sem exigir números idênticos.

Se não acontecer: Uma resposta vazia em repo recém-criado pode exigir aguardar o processamento de estatísticas do GitHub.

Passo 3

Terminal e caderno - encontre Handler.do_GET, search e sqlite3.execute. Escreva entrada, propagação, operação e controle.

```
nl -ba app.py
```

Esperado: Mapa: query q -> search -> execute; controle: parâmetro vinculado.

Se não acontecer: Se o relatório não expuser algum detalhe, marque “não observado”; não complete a evidência por suposição.

Passo 4

Caderno - conclua que esta amostra avalia Python/biblioteca padrão. Para outro framework, será necessário repetir a análise em código autorizado. Nenhuma limpeza de PR é necessária.

```
git status --short
```

Esperado: A árvore continua limpa e a conclusão tem escopo explícito.

Se não acontecer: Não declare cobertura de todos os frameworks com base neste único exercício.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Language and Framework Support, páginas 1-2.

SCANNING

09. Vulnerability Coverage Matrix

Mecanismo e decisões técnicas

A matriz documenta classes de vulnerabilidade, descrições e CWEs associados. Ela orienta um plano de avaliação, mas não promete detectar toda ocorrência de cada CWE em qualquer contexto. Algumas classes dependem de entender autorização de negócio; outras dependem de fluxo de dados, uso criptográfico ou configuração.

Organize a leitura em famílias: injeções e interpretação de entrada; autenticação/autorização; exposição de dados e segredos; SSRF e acesso a recursos; criptografia e configuração; lógica de negócio e uso de LLM. Entre as classes documentadas está LLM Tool Misuse and Prompt Injection, com CWE-1426 e CWE-1427. Essa cobertura deve ser avaliada com o código e os limites de confiança da integração, não apenas com o texto de um prompt.

Para modelagem de ameaça, use a matriz como lista de hipóteses. Descreva ativo, ator, entrada, fronteira de confiança, caminho de abuso e controle esperado; depois procure evidência com PR Scanning, DeepScan e Code Security Intelligence. O processo humano ainda define impacto de negócio e aceita ou rejeita risco.

Caso de uso

Um assistente lê tickets não confiáveis e pode chamar uma ferramenta que modifica permissões. O risco surge quando o conteúdo do ticket influencia uma ação privilegiada sem autorização independente. O review deve procurar o local onde texto vira ação, quais argumentos são validados e quem autoriza a ferramenta.

Antes de começar

Módulos 3 e 5; caderno disponível.

Laboratório guiado 09

Passo 1

Navegador - abra a matriz oficial na referência deste módulo. Localize SQL Injection e a categoria de uso indevido de ferramentas LLM/prompt injection. Endereço: https://docs.dryrun.security/vulnerability-coverage-matrix

Esperado: Você encontra a classe, descrição e CWE associado.

Se não acontecer: Não confunda um CWE, que descreve uma classe, com um finding ID da sua execução.

Passo 2

Terminal - crie um arquivo de hipóteses em .evidence. O texto distingue uma falha demonstrada de sistemas que nem existem neste repo.

```
source "$HOME/dryrun-student/session.sh"
mkdir -p .evidence
cat > .evidence/threats.txt <<'TXT'
TM01 SQLi: q -> search -> SQL. Demonstrated in lab 03.
TM02 IDOR: cross-user invoice access. Not implemented here.
TM03 LLM tool misuse: untrusted ticket -> tool. Not implemented here.
TXT
cat .evidence/threats.txt
```

Esperado: O arquivo tem três linhas de ameaça com estados diferentes.

Se não acontecer: Se .evidence não existir, mkdir -p a cria; ela é ignorada pelo Git.

Passo 3

Caderno - para TM01, registre ativo (produtos), entrada (q), fronteira (HTTP/SQL), controle (binding) e evidência (teste e scan). Para TM02/TM03, registre “hipótese não testada”.

Esperado: Você não cria tickets de vulnerabilidade confirmada para hipóteses sem implementação/evidência.

Se não acontecer: Se um scan estiver vazio, não marque todas as ameaças como resolvidas.

Passo 4

Caderno - escolha uma nova classe da matriz e descreva qual código seria necessário para testá-la. Salve o arquivo; não há PR para fechar.

```
git status --short
```

Esperado: O resultado é um plano de avaliação com limites, não um certificado de segurança.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Vulnerability Coverage Matrix, páginas 1-5.

PLATFORM

10. Reporting

Mecanismo e decisões técnicas

Reporting apresenta atividade e resultados em janelas como 24 horas, 7, 30 e 90 dias. As métricas incluem PRs analisadas, findings, PRs mescladas e risco levado ao merge. O indicador de melhoria acompanha PRs nas quais findings deixam de aparecer em scans seguintes; não representa uma auditoria manual de cada correção.

As unidades importam: quantidade de findings, de scans e de PRs são números diferentes. Uma PR pode ter vários scans e vários findings. Uma queda no número de achados pode resultar de menos atividade, mudança de escopo, ajuste de configuração ou correção real. A interpretação deve acompanhar o volume de PRs, a janela e a composição de repositórios.

A API de findings é útil para investigação e exportação, mas uma consulta a all_findings não reproduz automaticamente todos os agregados do dashboard. Filtros de branch, execuções mais recentes e estado alteram o conjunto retornado. Relatórios locais deste curso devem ser identificados como recortes próprios.

Caso de uso

Um gestor quer avaliar se o processo evita risco antes do merge. Compare PRs com findings corrigidos, PRs que foram mescladas com risco e motivos de exceção; não use apenas a contagem absoluta de vulnerabilidades para comparar equipes com volumes diferentes.

Antes de começar

Módulos 2 e 3 executados; permissão para visualizar Reporting.

Laboratório guiado 10

Passo 1

DryRun - abra Reporting e selecione 7 dias. Confira organização e filtros de repositório disponíveis. Não altere a configuração de scanning.

Esperado: A janela e o escopo ficam visíveis para você registrar.

Se não acontecer: Se não houver dados, tente 30 dias e confira a organização; não invente atividade para preencher a tabela.

Passo 2

Terminal - prepare o registro do relatório; depois preencha com os números e unidades exibidos no painel.

```
source "$HOME/dryrun-student/session.sh"
cat > .evidence/report.txt <<'TXT'
Observed at / time zone:
Window:
Repository scope:
Scanned PRs:
Findings:
Merged PRs with risk:
Configuration changes:
Interpretation and limitations:
TXT
```

Esperado: Você tem uma ficha sem afirmar que os valores ainda não preenchidos são zero.

Se não acontecer: Campo indisponível deve ser marcado “não disponível”.

Passo 3

GitHub - liste suas PRs e compare uma PR do curso com o painel. Uma PR pode ter mais de um scan.

```
gh pr list --state all --limit 30
```

Esperado: Você consegue explicar por que contagem de scans não precisa ser igual à de PRs.

Se não acontecer: A listagem usa um limite de 30; não a apresente como histórico completo se houver mais PRs.

Passo 4

Caderno - escreva uma interpretação e uma limitação. Exemplo: exercícios corrigidos antes do encerramento não representam merges de produção. Salve; não há PR para fechar.

Esperado: Relatório contém fonte, janela, unidade e limitação.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Reporting, páginas 1-4; referência API all_findings consultada em 20/10/2026.

PLATFORM

11. Risk Register

Mecanismo e decisões técnicas

Risk Register é o local de trabalho para investigar e acompanhar findings. Ele reúne o que foi detectado em diferentes superfícies de análise e permite usar filtros e estados para organizar tratamento. A decisão deve preservar a explicação técnica: o status sozinho não informa por que um finding foi considerado falso positivo ou aceito.

Separe correção de triagem. Resolved indica uma decisão/estado de tratamento; um novo scan é a evidência necessária para demonstrar que o caminho vulnerável desapareceu. Accepted Risk registra uma aceitação e não equivale a falha inexistente. False Positive significa que o raciocínio não se aplica ao código naquele contexto. Won't Fix/Nitpick comunica que não haverá correção, com justificativa apropriada.

O estado afeta workflows de formas diferentes. Na documentação de blocking, apenas False Positive e Won't Fix/Nitpick liberam automaticamente um check de falha relacionado à triagem. Accepted Risk, In Progress e Resolved não o liberam. Se sua política exige uma exceção formal de merge, ela deve ser tratada no processo de governança, não inferida pelo nome do status.

Caso de uso

Um finding SQLi real é atribuído para correção. Marcá-lo como In Progress ajuda o acompanhamento, mas o check continua falhando até que a condição de bloqueio seja removida. Uma discussão sobre prazo não muda o comportamento técnico do código.

Antes de começar

Módulo 3; se não houver finding, registre o laboratório de triagem como não observado.

Laboratório guiado 11

Passo 1

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/11
```

Esperado: A última mensagem deve indicar a branch lab/11. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 2

Terminal - crie uma ocorrência nova para praticar triagem sem reutilizar um finding já corrigido.

```
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m 'Lab: finding for triage'
```

Esperado: A branch contém a SQLi do módulo 3.

Passo 3

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 11 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 4

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 5

DryRun - abra Risk Register, filtre pelo seu repo/PR e abra o finding. Copie o ID no caderno. Escolha In Progress e escreva uma justificativa verdadeira: teste demonstra concatenação SQL; correção será parametrização.

Esperado: A decisão aparece no histórico/status. Isso ainda não corrige o código.

Se não acontecer: Se o finding não aparecer, confira filtros e execução. Não marque uma SQLi real como False Positive para completar a atividade.

Passo 6

Terminal - aplique a correção, teste e envie. Depois retorne ao finding e compare o novo scan com o registro de triagem.

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Repair finding under investigation'
git push
```

Esperado: Os testes passam e a evidência de reparo fica separada da decisão administrativa.

Passo 7

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 8

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Risk Register, páginas 1-4; PR Blocking, páginas 1-6.

PLATFORM

12. Code Security Intelligence

Mecanismo e decisões técnicas

Code Security Intelligence permite investigar o código e o histórico com perguntas em linguagem natural. A documentação descreve workflows como Feature Ships, Resolved Risk, Vulnerability Trends, Architecture Risks, Developer Trends, Shadow AI, Incident Response, Application Summary, Security Reviews, New Feature Review e PR Variant Analysis.

A qualidade de uma pergunta depende do escopo: organização, repo, branch, janela e objetivo. Peça que cada conclusão cite arquivo/linha, PR, commit ou finding quando disponível. Se a ferramenta não tiver dados suficientes, a resposta deve separar observação de hipótese. Um texto fluente não substitui evidência.

Para modelagem de ameaça, CSI ajuda a levantar pontos de entrada, ativos, fluxos de dados e fronteiras de confiança. O analista revisa essa proposta e decide quais hipóteses exigem testes. O workflow de variantes pode procurar padrões relacionados a um bug; não deve declarar a erradicação de uma classe inteira com base em uma única correção.

Caso de uso

Após corrigir SQLi, o time quer saber se outras consultas usam concatenação e se uma biblioteca interna implementa parametrização. Uma consulta específica com evidências produz uma lista revisável para investigação; “o sistema está seguro?” tende a gerar uma resposta pouco acionável.

Antes de começar

Módulos 5 e 9; acesso a Code Security Intelligence.

Laboratório guiado 12

Passo 1

DryRun - abra Code Security Intelligence e selecione seu repo. Escolha Application Summary ou um fluxo de consulta equivalente disponível.

Esperado: O escopo mostra o repo de exercícios, não todos os repositórios da organização.

Se não acontecer: Se o recurso não estiver disponível, registre a limitação de acesso; não atribua uma resposta genérica ao CSI.

Passo 2

Caixa de pergunta do DryRun - cole o texto abaixo como pergunta, não como comando de terminal.

```
PROMPT:
Map HTTP entry points and SQL operations in this repository.
Cite file/line and the analyzed reference for each conclusion.
Explain how query input reaches SQL and what control applies.
Separate observed facts, hypotheses and unavailable evidence.
Do not modify code, findings or configuration.
```

Esperado: A resposta deve indicar evidências ou declarar que não as possui.

Se não acontecer: Se ficar genérica, acrescente o caminho app.py e peça uma resposta limitada a essa implementação.

Passo 3

Terminal - confira pelo menos uma linha citada. Compare a resposta ao modelo de ameaça do módulo 9.

```
nl -ba app.py
cat .evidence/threats.txt
```

Esperado: Você confirma ou corrige uma afirmação e anota o motivo.

Se não acontecer: Uma citação que não corresponde ao arquivo pode indicar referência/branch diferente. Confira antes de confiar.

Passo 4

DryRun e caderno - salve a consulta se a interface permitir, com nome “Training - SQL boundary”. Registre uma hipótese ainda não testada. Nenhuma PR é criada.

Esperado: Resultado: consulta delimitada, evidência verificada e limitação explícita.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Code Security Intelligence, páginas 1-4.

PLATFORM

13. PR Scanning Configuration

Mecanismo e decisões técnicas

A Default Configuration governa repositórios sem uma configuração salva específica. Ao atribuir um repo a uma Saved Configuration, ele passa a ser governado exclusivamente por ela. Um repo pertence a uma configuração por vez. Não trate isso como duas regras independentes que se somam em execução.

As abas separam comentários, blocking, notificações, contexto, políticas e agentes. Por padrão documentado, os agentes estão habilitados, blocking está desligado, o limiar é High, comentários estão ligados e notificações desligadas. Salvar a configuração passa a afetar o próximo scan; não reinterpreta automaticamente todo o histórico.

Silent mode oculta a saída do agente/política no comentário, mas mantém análise, registro e eventual check de falha. Desligar Issue Comments também não desliga o scanner. Blocking geral usa severidade dos Code Security Agents; blocking por agente ou política é binário: qualquer finding daquela origem pode falhar independentemente da severidade.

Deduplicate Notifications compara a maior severidade do scan atual com a do anterior da mesma PR. Ela não faz deduplicação por finding ID. Um novo finding High após um High pode não gerar novo alerta; uma elevação de High para Critical pode gerar. A configuração aceita até sete políticas.

Caso de uso

Um time quer testar um agente sem comentários no GitHub. Silent mode permite observar os findings no Risk Register. Se Blocking permanecer ativo, o time poderá receber check de falha mesmo sem explicação no comentário; isso deve ser intencional e comunicado.

Antes de começar

Módulo 2; Admin para criar configuração exclusiva do repo de exercícios.

Laboratório guiado 13

Passo 1

DryRun - abra Settings > Configuration. Crie uma Saved Configuration chamada Training-observe e associe somente seu repo de exercícios. Anote a configuração anterior antes de alterar.

Esperado: O repo está associado a uma configuração salva própria.

Se não acontecer: Não altere a Default da organização para este exercício. Se já houver Training-observe, confira seu escopo antes de reutilizar.

Passo 2

DryRun - em General, habilite Issue Comments e Show comment for no findings; deixe PR blocking desligado. Em Notifications, deixe notificações desligadas. Salve.

Esperado: Você consegue listar os valores salvos. O próximo scan é que usará esses valores.

Se não acontecer: Se Save falhar, não continue supondo que as opções foram aplicadas.

Passo 3

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/13
```

Esperado: A última mensagem deve indicar a branch lab/13. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 4

Terminal - faça uma mudança segura para gerar o próximo scan.

```
printf '\nConfiguration exercise.\n' >> README.md
git add README.md
git commit -m 'Exercise scan configuration'
```

Esperado: Um commit de documentação foi criado.

Passo 5

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 13 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 6

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 7

DryRun - desligue Issue Comments na configuração de treinamento e salve. Terminal - envie outra mudança segura. Confira que o painel/check continua registrando a análise.

```
printf '\n# Configuration observation.\n' >> app.py
git add app.py
git commit -m 'Observe scan without comments'
git push
```

Esperado: Desligar comentários não desliga scanning. Um comentário antigo pode pertencer ao scan anterior.

Se não acontecer: Compare o SHA e o horário; não interprete comentário antigo como configuração ignorada.

Passo 8

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 9

DryRun - reative comentários e salve. Mantenha Training-observe para os próximos módulos, com blocking e notificações desligados.

Esperado: A configuração final fica conhecida e documentada.

Passo 10

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: PR Scanning Configuration, páginas 1-10.

PLATFORM

14. Custom Code Policies

Mecanismo e decisões técnicas

Custom Code Policies expressam regras específicas em linguagem natural. Diferentemente dos agentes gerais, elas podem representar restrições de arquitetura ou de engenharia da organização. Uma boa política define comportamento proibido/obrigatório, escopo, exceções e exemplos suficientes para reduzir interpretações amplas.

Cada política pode receber severidade uniforme, Silent mode e Blocking ao ser anexada à configuração. Blocking de política é binário; um finding Low pode falhar se essa política estiver configurada para bloquear. O Background da política deve conter o contexto necessário: políticas não herdam automaticamente Repository Context nem AGENTS.md.

Evite instruções vagas como “siga as melhores práticas”. Prefira uma condição observável, por exemplo, não usar subprocess com shell=True quando o comando contém entrada de requisição. Uma regra assim exige investigar a origem da entrada, não proibir toda chamada a subprocess em scripts administrativos controlados.

Caso de uso

A organização proíbe eval em código Python de aplicação, inclusive quando um desenvolvedor acredita que os dados são internos. A política torna essa convenção explícita e repetível em PRs, sem esperar que uma regra genérica coincida com a decisão local.

Antes de começar

Módulo 13; permissão para criar política.

Laboratório guiado 14

Passo 1

DryRun - abra Custom Code Policies e crie “Training - no eval”. Preencha a regra e Background com o texto abaixo nos campos correspondentes.

```
RULE:
Report calls to Python eval() in application source files.
Ignore documentation and test fixtures.
Recommend explicit parsing or an allowlisted dispatch table.
BACKGROUND:
calculator.py is application code. test_* files are tests.
```

Esperado: A regra tem comportamento, escopo e exceção explícitos.

Se não acontecer: Não cole RULE/BACKGROUND como nomes de API; são rótulos didáticos dos campos da interface.

Passo 2

DryRun - em Training-observe > Code Policies, associe a política. Selecione Medium, Blocking off e Silent mode off. Salve.

Esperado: A política aparece na configuração que governa seu repo.

Se não acontecer: Criar a política sem associá-la não basta. Confira também o limite de sete políticas.

Passo 3

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/14
```

Esperado: A última mensagem deve indicar a branch lab/14. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 4

Terminal - crie a violação, salve e publique no passo seguinte. Não execute eval com entrada externa.

```
cat > calculator.py <<'PYCODE'
def calculate(expression):
    return eval(expression)
PYCODE
git add calculator.py
git commit -m 'Lab: eval policy violation'
```

Esperado: calculator.py contém uma chamada eval.

Passo 5

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 14 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 6

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 7

Terminal - substitua o exemplo por soma explícita, confira a sintaxe e envie. No DryRun, diferencie o finding de política de um finding genérico.

```
cat > calculator.py <<'PYCODE'
def calculate_sum(left, right):
    return float(left) + float(right)
PYCODE
python3 -m py_compile calculator.py
git add calculator.py
git commit -m 'Replace eval with explicit operation'
git push
```

Esperado: A chamada proibida não existe mais; compile termina sem erro.

Se não acontecer: Um erro de sintaxe precisa ser corrigido antes do push.

Passo 8

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 9

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Passo 10

DryRun - desassocie a política experimental de Training-observe e salve, para não influenciar os próximos módulos.

Esperado: A política pode permanecer cadastrada como referência, mas não está ativa no repo.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Custom Code Policies, páginas 1-5; PR Scanning Configuration, páginas 7-8.

PLATFORM

15. Repository Context

Mecanismo e decisões técnicas

Repository Context fornece informações que não são facilmente inferidas do código: autenticação compartilhada, contratos de bibliotecas internas, arquitetura, modelo de tenancy e controles externos. O contexto deve descrever fatos verificáveis. Dizer “todas as entradas são seguras” sem apontar onde e como são validadas pode esconder uma falha real.

Em PRs, pode-se associar documentos pela configuração e usar AGENTS.md na raiz do repositório. DeepScan também descobre AGENTS.md em subdiretórios. Se houver contexto específico configurado para DeepScan, ele substitui os AGENTS.md naquela análise. O alcance dos arquivos e essa precedência devem ser conhecidos para evitar resultados aparentemente contraditórios.

Contexto não é feedback de um finding individual. Use contexto para invariantes da arquitetura e triagem para a decisão sobre uma ocorrência concreta. Custom Code Policies precisam de seu próprio Background. Além disso, AGENTS.md pode ser lido por outros agentes de desenvolvimento: seu texto deve ser correto e apropriado para esses consumidores também.

Caso de uso

Uma biblioteca interna faz autorização por tenant antes de chamar handlers. Documentar o wrapper e o ponto onde a identidade é validada permite que a investigação verifique o controle. Uma alegação genérica de que “a aplicação fica atrás de um gateway” não prova autorização por objeto.

Antes de começar

Módulos 2 e 13; acesso a Repository Context.

Laboratório guiado 15

Passo 1

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/15
```

Esperado: A última mensagem deve indicar a branch lab/15. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 2

Terminal - crie um AGENTS.md na raiz com fatos da aplicação. O texto diz explicitamente que não existe autenticação.

```
cat > AGENTS.md <<'TXT'
# Training security context
app.py exposes GET /search?q= on loopback only.
There is no authentication or tenant authorization middleware.
SQLite is in memory and contains synthetic products.
SQL input must remain a bound value, not an SQL fragment.
Do not assume that a gateway validates input.
TXT
git add AGENTS.md
git commit -m 'Add verifiable repository context'
```

Esperado: Cada linha pode ser confrontada com app.py.

Se não acontecer: Não escreva uma proteção fictícia só para reduzir findings.

Passo 3

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 15 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 4

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 5

DryRun - abra Repository Context, crie um documento “Training context” com os mesmos fatos e salve. Em Training-observe > Context, selecione esse documento e salve novamente.

Esperado: O documento existe e está associado ao repo pela configuração.

Se não acontecer: Se apenas criar o documento, ele pode não estar associado a scan algum.

Passo 6

Terminal - envie um comentário seguro para testar o próximo scan com a configuração salva.

```
printf '\n# Context exercise.\n' >> app.py
git add app.py
git commit -m 'Observe configured context'
git push
```

Esperado: O novo scan usa o estado posterior à associação.

Passo 7

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 8

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Passo 9

DryRun - desassocie o contexto experimental ao terminar ou mantenha apenas fatos válidos para main. Registre a decisão.

Esperado: Você sabe qual contexto continuará ativo.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Repository Context, páginas 1-4.

PLATFORM

16. Finding Tuning with Feedback

Mecanismo e decisões técnicas

A documentação descreve aprendizado a partir de triagens False Positive, Won't Fix e Accepted Risk quando acompanhadas de explicação útil. Uma resposta vazia ou “noted” não oferece um precedente aplicável. Para findings de PR, o aprendizado ocorre depois do merge; para DeepScan, é imediato. In Progress e Resolved não são categorias de aprendizado.

Isso é diferente de liberar um check. Accepted Risk pode fornecer contexto para aprendizado, mas não libera automaticamente a falha de PR Blocking. False Positive e Won't Fix/Nitpick têm esse efeito de liberação documentado. O texto da decisão precisa deixar claro qual situação foi aceita e quais controles existem.

Um bom feedback descreve por que o fluxo é seguro ou por que o risco é aceito, aponta arquivos/controles e limita a generalização. Um feedback ruim afirma “falso positivo” porque a correção é inconveniente. Não use uma falha real de laboratório para ensinar ao scanner que concatenação SQL é segura.

Caso de uso

Um wrapper interno faz validação e autorização antes de uma operação marcada como suspeita. O analista confirma a implementação e fornece a referência concreta. Se a informação se aplica a vários achados e repositórios, complementa Repository Context em vez de repetir uma justificativa vaga em cada triagem.

Antes de começar

Módulo 11. Parte de publicação só deve ser executada para um falso positivo realmente confirmado; sem isso, conclua a simulação local.

Laboratório guiado 16

Passo 1

Terminal - prepare um rascunho de feedback sem enviar nada.

```
source "$HOME/dryrun-student/session.sh"
cat > .evidence/feedback.txt <<'TXT'
Finding ID:
Proposed category:
Scanner hypothesis:
Verified control:
File, line and commit:
Why this control applies:
Limits of this explanation:
TXT
```

Esperado: O arquivo é local e não altera aprendizado.

Passo 2

Caderno - use o finding real de SQLi como exemplo: a categoria correta para trabalho em andamento é In Progress, não FP. Escreva o motivo e explique por que a falha é verdadeira.

Esperado: Você identifica quando não deve enviar feedback de falso positivo.

Se não acontecer: Se não conseguir provar um controle, mantenha a investigação aberta.

Passo 3

Opcional, GitHub - somente se você tiver outro finding realmente falso, escreva o comentário final em .evidence/feedback-final.txt no formato abaixo, substituindo o ID e a justificativa.

```
COMMENT FORMAT:
@dryrunsecurity FP FINDING_ID verified technical explanation
```

Esperado: O arquivo final não contém placeholders.

Se não acontecer: Não envie o formato de exemplo literalmente. Sem um caso real, pule a publicação.

Passo 4

Opcional, Terminal - confira a PR correta antes de publicar o arquivo revisado. Os comandos abaixo só se aplicam ao caso confirmado do passo anterior.

```
read -rp 'Verified target PR number: ' TARGET_PR
gh pr view "$TARGET_PR" --web
cat .evidence/feedback-final.txt
# Run only after reviewing the real justification:
# gh pr comment "$TARGET_PR" --body-file .evidence/feedback-final.txt
```

Esperado: A atividade obrigatória termina com um rascunho correto; publicação depende de evidência real.

Se não acontecer: Não mescle código vulnerável para provocar aprendizado. PR aprende após merge; DeepScan aprende imediatamente.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Finding Tuning with Feedback, páginas 1-6; PR Blocking, páginas 1-6.

PLATFORM

17. PR Blocking

Mecanismo e decisões técnicas

Bloqueio efetivo combina configuração DryRun e enforcement do SCM. DryRun envia check de falha quando a regra é satisfeita; o GitHub deve exigir esse check para impedir merge. Sem essa segunda parte, o check vermelho pode ser apenas informativo. Uma configuração do DryRun não cria automaticamente todas as proteções necessárias no GitHub.

Há blocking geral por severidade, blocking por agente e blocking por política. Os dois últimos são binários. No fluxo de rulesets de organização, a documentação orienta o check DryRun com origem DryRunSecurity. Configurações clássicas podem apresentar checks específicos de agentes/políticas. Descubra os nomes e app IDs efetivos na sua PR antes de criar uma regra.

GitHub Rulesets pode centralizar enforcement por organização. O laboratório abaixo usa proteção clássica de uma branch em um repo dedicado para tornar o payload revisável. A disponibilidade depende do plano e das permissões do GitHub. No GitLab, a opção Pipelines must succeed afeta o resultado do pipeline, não somente DryRun.

Caso de uso

A equipe define que High/Critical impedem merge. Uma SQLi High gera falha, mas o usuário ainda consegue mesclar porque main não exige o check. O problema está no enforcement, não na análise da vulnerabilidade.

Antes de começar

Módulo 13; administrador do repo de exercícios; plano GitHub que permita a regra. Não aplique este exercício a uma branch de produção.

Laboratório guiado 17

Passo 1

DryRun - em Training-observe > General, ative PR blocking com Minimum blocking risk High e salve. Deixe bloqueios binários adicionais desligados para isolar o teste.

Esperado: A configuração está salva apenas para o repo de exercícios.

Se não acontecer: Se o repo não for exclusivo, volte ao módulo 13 antes de continuar.

Passo 2

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/17
```

Esperado: A última mensagem deve indicar a branch lab/17. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 3

Terminal - introduza e comite a SQLi.

```
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m "Lab: blocking condition"
```

Esperado: A alteração está nesta branch, não em main.

Passo 4

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 17 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 5

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 6

Terminal - descubra o nome e a App do check real. Copie o nome exato que corresponde ao DryRun, não o de outro CI.

```
SHA=$(gh pr view --json headRefOid --jq .headRefOid)
gh api "repos/$GH_REPO/commits/$SHA/check-runs" \
  --jq '.check_runs[] | {name,app:.app.slug,conclusion}'
```

Esperado: Você identifica contexto, App e conclusão do scan atual.

Se não acontecer: Se a lista estiver vazia, volte à instalação/execução; não crie uma regra para um nome inventado.

Passo 7

GitHub - no repo de exercícios, abra Settings > Rules > Rulesets e crie um branch ruleset “Training-DryRun”. Selecione main como alvo, enforcement Active, Require status checks to pass e o check DryRun observado. Escolha a App de origem quando o seletor permitir. Não adicione bypass para o aluno. Salve.

Esperado: Na PR com falha, o GitHub indica que o check obrigatório impede merge. Não clique em Merge.

Se não acontecer: Se a opção não existir, peça ao administrador para confirmar plano/permissão. Não substitua uma regra existente. Se a severidade for menor que High, registre que o limiar não foi atingido.

Passo 8

Terminal - corrija a consulta e envie; confira novamente o check e a disponibilidade de merge sem realizar o merge.

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Remove blocking vulnerability'
git push
```

Esperado: O novo scan avalia o patch. Um check aprovado remove essa condição de bloqueio; outras regras podem continuar pendentes.

Passo 9

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 10

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Passo 11

GitHub - desative ou exclua somente o ruleset Training-DryRun criado neste lab. DryRun - desligue blocking de Training-observe e salve.

Esperado: A limpeza não remove proteções preexistentes.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: PR Blocking, páginas 1-6; GitHub REST Branch protection.

PLATFORM

18. Permissions

Mecanismo e decisões técnicas

DryRun distingue Admin e Developer. A documentação associa administradores do GitHub ao papel Admin e membros ao papel Developer; no GitLab, Owner/Maintainer correspondem ao papel administrativo descrito. O acesso aos dados também depende do acesso aos repositórios no provedor.

Administradores podem gerir configurações, políticas e chaves e iniciar recursos administrativos como DeepScan e Insights. Desenvolvedores consultam os repositórios autorizados e podem participar da triagem conforme a funcionalidade disponível. A autorização de um token não deve ser presumida apenas porque a pessoa consegue abrir o dashboard: o token tem seu próprio escopo.

O desenho operacional deve separar quem investiga findings, quem altera critérios de bloqueio e quem administra integrações. Isso evita que uma mudança de visibilidade ou de gate passe despercebida durante uma correção urgente. O laboratório não altera permissões de pessoas na organização.

Caso de uso

Uma pessoa consegue ver a PR no GitHub, mas uma chamada da API DryRun retorna 403. Investigue organização, instalação, papel, escopo da chave e repo solicitado. Trocar por uma chave de administrador pode mascarar o problema e ampliar acesso desnecessariamente.

Antes de começar

Módulo 2. Não é necessário criar usuários nem compartilhar tokens.

Laboratório guiado 18

Passo 1

Terminal - consulte suas permissões no repo GitHub. Esse resultado é do GitHub, não uma lista completa das permissões DryRun.

```
source "$HOME/dryrun-student/session.sh"
gh api "repos/$GH_REPO" --jq '{full_name,permissions}'
```

Esperado: Registre o nome do repo e os valores retornados.

Passo 2

DryRun - confira papel e organização. Tente localizar, sem alterar, configuração, Access Keys e DeepScan. Anote o que está disponível.

Esperado: Você distingue ações visíveis de ações realmente autorizadas.

Se não acontecer: Não peça a senha ou token de outra pessoa para comparar.

Passo 3

Caderno - monte uma matriz com ação, papel observado, permissão esperada e evidência. Se houver usuário de teste já autorizado, compare via sessão separada; caso contrário, marque essa comparação como não executada.

Esperado: A matriz separa permissão SCM, papel DryRun e escopo de chave.

Se não acontecer: 401 normalmente aponta credencial; 403, autorização; 404 também pode ocultar recurso sem acesso.

Passo 4

Caderno - registre o acesso adicional mínimo que seria necessário para um lab bloqueado. Não altere permissões reais só para testar.

```
git status --short
```

Esperado: Nenhuma alteração de usuário ou repo foi feita.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Permissions, páginas 1-3; API Access Keys, páginas 1-2.

PLATFORM

19. MCP

Mecanismo e decisões técnicas

O servidor MCP do DryRun fornece ferramentas para um cliente de IA consultar o contexto de segurança. O endpoint documentado é https://insights-mcp.dryrun.security/api/insights/mcp e a autenticação utiliza Bearer token. O cliente precisa iniciar a sessão MCP e descobrir as ferramentas; fazer um GET comum com curl não substitui uma conversa MCP completa.

Separe os componentes: o cliente Codex interpreta o pedido; o servidor MCP expõe as operações e dados autorizados; o DryRun é a fonte de contexto; o repositório local é onde uma eventual correção será preparada. A chave determina o acesso. Nomes e schemas das ferramentas devem ser descobertos pelo cliente, não adivinhados em um script.

Os PDFs mostram formatos genéricos de configuração para clientes. No Codex, a CLI abaixo registra o servidor HTTP e referencia uma variável de ambiente. Isso evita gravar o token literal no arquivo de configuração. Execute o Codex em um terminal que tenha a variável exportada.

Caso de uso

Um analista pede ao Codex os findings do repo e a explicação do caminho vulnerável. O assistente consulta DryRun e relaciona as evidências com os arquivos locais. Uma etapa posterior, explicitamente solicitada, prepara a correção. O laboratório inicial permanece em leitura para validar a conexão.

Antes de começar

Módulo 2; Codex instalado/autenticado conforme preparação; execute a preparação “Chave DryRun” antes deste módulo.

Laboratório guiado 19

Passo 1

Terminal - carregue a sessão e verifique a autenticação Codex e a presença da variável, sem imprimir a chave.

```
source "$HOME/dryrun-student/session.sh"
codex login status
test -n "$DRYRUN_API_KEY" && echo 'DryRun key is loaded'
```

Esperado: Você vê autenticação válida e “DryRun key is loaded”.

Se não acontecer: Sem a mensagem da chave, repita a preparação Chave DryRun neste terminal.

Passo 2

Terminal - registre um servidor chamado dryrun-training. O nome é exclusivo do curso para não substituir uma conexão pessoal existente.

```
codex mcp add dryrun-training \
  --url https://insights-mcp.dryrun.security/api/insights/mcp \
  --bearer-token-env-var DRYRUN_API_KEY
codex mcp get dryrun-training
```

Esperado: A configuração aponta à URL correta e ao nome da variável, não ao token literal.

Se não acontecer: Se o nome já existir, confira codex mcp get antes de mudar algo. Não use login OAuth como substituto automático de Bearer.

Passo 3

Terminal - abra o Codex em modo de leitura. Dentro dele, use /mcp para conferir a conexão.

```
codex --sandbox read-only
```

Esperado: A conexão deve disponibilizar ferramentas ou informar o erro específico.

Se não acontecer: 401 pode indicar chave inválida; escopo/permissão ou variável não herdada também devem ser conferidos.

Passo 4

Chat do Codex - peça: “Descubra as ferramentas DryRun. Consulte somente meu repo OWNER/REPO. Cite IDs e fontes. Não altere código, triagem ou configuração.” Troque OWNER/REPO pelo nome real.

Esperado: Uma chamada real funciona e a resposta corresponde ao repo. Zero findings é diferente de falha de conexão.

Se não acontecer: Se o agente não puder consultar, registre a limitação em vez de aceitar uma resposta inventada.

Passo 5

Codex - encerre a sessão interativa com Ctrl+C quando não houver operação em andamento. De volta ao terminal, confira o estado local. Mantenha a conexão para o módulo 24.

```
git status --short
```

Esperado: Nenhuma alteração local foi criada.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: MCP, páginas 1-4; comandos Codex conferidos na referência oficial Developer commands.

PLATFORM

20. DryRun API

Mecanismo e decisões técnicas

A Simple API usa a base https://simple-api.dryrun.security/v1. A maioria dos recursos fica sob account_id e, quando necessário, repository_id. O fluxo de descoberta é conta -> repositório -> execução ou finding. Os IDs são valores retornados pela API; não substitua um UUID por org/repo ou por número da PR.

O endpoint repositories/{RID}/findings lista findings de PR. Para combinar PR, DeepScan, SCA e políticas, use all_findings e os filtros correspondentes. O parâmetro days aceita a janela documentada de 1 a 365 dias. A consulta pode retornar apenas a página solicitada e, conforme os filtros/defaults, somente os resultados mais recentes; registre esses critérios ao exportar.

Requisições de leitura podem ser repetidas para diagnóstico. POST de DeepScan inicia trabalho e retorna aceitação, não conclusão. Se uma chamada de escrita perder a resposta, confira o estado antes de repetir para não gerar scans duplicados. Um HTTP 202 não comprova que todas as etapas concluíram.

Os PDFs listam endpoints de configuração e triagem, mas não imprimem todos os schemas expandidos. Para alterações desses recursos, consulte o Swagger/OpenAPI da versão atual e use a interface quando o contrato não estiver claro. O curso não inventa campos de payload para preencher essa lacuna.

Caso de uso

Você precisa exportar os findings de um repo e correlacioná-los com uma execução para criar tickets. Consulta a lista, guarda IDs e detalhes e só então transforma o resultado. Se o integrador tratar todo retorno como array simples, pode perder metadados de paginação ou falhar ao receber um erro JSON.

Antes de começar

Módulo 2; preparação Chave DryRun. Para o POST opcional, o token deve permitir iniciar DeepScan.

Laboratório guiado 20

Passo 1

Terminal - configure a base, faça a primeira leitura e confira o JSON. dryget é um auxiliar do curso que usa curl com Authorization Bearer.

```
source "$HOME/dryrun-student/session.sh"
dryget accounts > .evidence/accounts.json
jq . .evidence/accounts.json
```

Esperado: Há uma lista data com contas acessíveis.

Se não acontecer: Se curl falhar, pare e leia o erro. Não use um arquivo vazio como resposta válida.

Passo 2

Terminal - localize a conta certa pelo org_name, copie seu account_id e cole quando solicitado. Depois liste os repos e escolha o id do seu repo de exercícios.

```
read -rp 'account_id: ' AID
export AID
dryget "accounts/$AID/repositories" > .evidence/repositories.json
jq . .evidence/repositories.json
read -rp 'repository id: ' RID
export RID
```

Esperado: AID e RID vêm das respostas; não são exemplos deste PDF.

Se não acontecer: Se 404 ocorrer, confira conta, ID e acesso; não troque UUID por nome do repo.

Passo 3

Terminal - exporte scans e findings. Inspecione a estrutura antes de contar.

```
dryget "accounts/$AID/repositories/$RID/scans" > .evidence/scans.json
dryget "accounts/$AID/repositories/$RID/findings" > .evidence/pr-findings.json
QUERY="days=30&repository_id=$RID&per_page=100&page=1"
dryget "accounts/$AID/all_findings?$QUERY" > .evidence/all-findings-1.json
jq . .evidence/all-findings-1.json
```

Esperado: Você identifica resultados e metadados de paginação.

Se não acontecer: Se houver página seguinte, repita page=2 e salve outro arquivo. Uma página não prova que todo o histórico foi exportado.

Passo 4

Opcional, Terminal - somente se quiser iniciar um DeepScan novo e tiver permissão, execute o POST abaixo uma vez. Para escolher branch, prefira a interface do módulo 5; não adicionamos parâmetro branch não confirmado.

```
curl --fail-with-body --silent --show-error --request POST \
  -H "Authorization: Bearer $DRYRUN_API_KEY" \
  "$DRYRUN_BASE/accounts/$AID/repositories/$RID/deepscans" \
  > .evidence/start-deepscan.json
cat .evidence/start-deepscan.json
dryget "accounts/$AID/repositories/$RID/deepscans"
```

Esperado: Uma aceitação/execução em fila é apenas início; aguarde conclusão no painel.

Se não acontecer: Não repita POST para consultar progresso. Use GET/listagem.

Passo 5

Terminal - escolha o ID de uma execução concluída da lista. Exporte resultados e SBOM; confira se o formato retornado é JSON antes de processar com jq.

```
read -rp 'Completed DeepScan ID: ' SID
BASE="accounts/$AID/repositories/$RID/deepscans/$SID"
dryget "$BASE/results" > .evidence/deepscan-results.json
dryget "$BASE/sca_results" > .evidence/sca-results.json
dryget "$BASE/sbom" > .evidence/sbom.json
```

Esperado: Os arquivos são vinculados ao mesmo scan ID.

Se não acontecer: 401: credencial; 403: autorização; 404: ID/acesso; 429: aguarde conforme Retry-After; 5xx: registre request_id, se houver.

Passo 6

Caderno - salve IDs, filtros, janela e número de páginas exportadas. Não envie tokens nem respostas sensíveis ao repo do curso.

```
git status --short
```

Esperado: Os arquivos em .evidence continuam fora do versionamento.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: DryRun API, páginas 1-6; Swagger UI/OpenAPI oficial.

API INTEGRATIONS

21. Webhooks

Mecanismo e decisões técnicas

Webhooks transportam notificações de findings de PR para um endpoint. Crie o destino nas configurações de Webhooks e depois selecione-o na configuração de PR, habilitando notificações e limiar. Criar um endpoint sem associá-lo ao fluxo não garante eventos. A documentação também descreve Slack e daily digest; o digest tem finalidade diferente do evento individual.

O payload genérico contém metadados do repo/PR e uma lista results. Cada item pode conter id, type, filename, line_start, line_end, description, risk e finding_type. Consuma o evento real recebido antes de fechar um contrato de integração. O enum de um campo não é garantia de que todos os tipos de scan emitam esse webhook: a documentação de entrega descreve PRs, não um stream completo de DeepScan/SCA.

O receiver do curso captura o JSON e não cria tickets automaticamente. Um hash do conteúdo evita salvar duas cópias de payloads idênticos. Isso não é deduplicação por finding, pois eventos com timestamps diferentes produzem hashes diferentes. Em produção, use fila, persistência, autenticação compatível com o emissor, observabilidade, retries e identidade de finding para idempotência.

Caso de uso

A equipe precisa validar a forma real da notificação antes de implementar um fluxo Jira. Capturar um evento e compará-lo ao schema evita implementar o parser para um exemplo simplificado que não corresponde ao ambiente.

Antes de começar

Módulos 2 e 13. Requer três terminais para entrega externa: A receptor, B túnel, C comandos. cloudflared só é necessário para a parte externa.

Laboratório guiado 21

Passo 1

Terminal A - carregue a sessão, gere um segredo de caminho e inicie o receptor. Anote o segredo exibido para colá-lo no Terminal C. Este processo fica aberto; não espere o prompt retornar.

```
source "$HOME/dryrun-student/session.sh"
export WEBHOOK_PATH_KEY=$(python3 -c \
  'import secrets; print(secrets.token_urlsafe(32))')
printf '%s\n' "$WEBHOOK_PATH_KEY"
python3 "$KIT/receiver.py"
```

Esperado: O receptor informa que está ouvindo em 127.0.0.1:8080.

Se não acontecer: Address already in use: outro processo ocupa 8080. Encerre apenas o receptor anterior do curso.

Passo 2

Terminal C - abra outra janela do Terminal, carregue a sessão e cole o mesmo segredo quando solicitado. Envie a amostra local.

```
bash
source "$HOME/dryrun-student/session.sh"
read -rsp 'Path secret from terminal A: ' WEBHOOK_PATH_KEY; echo
export WEBHOOK_PATH_KEY
curl --fail-with-body -H 'Content-Type: application/json' \
  --data-binary "@$KIT/event-example.json" \
  "http://127.0.0.1:8080/hook/$WEBHOOK_PATH_KEY"
ls .evidence/events
```

Esperado: Resposta accepted:true e arquivo JSON em .evidence/events.

Se não acontecer: Connection refused: receptor parado. 404: segredo/caminho diferente. 400: JSON inválido.

Passo 3

Terminal B - instale cloudflared pela preparação e execute o túnel. Anote a URL https://...trycloudflare.com mostrada. Deixe o processo aberto.

```
cloudflared tunnel --url http://127.0.0.1:8080
```

Esperado: A URL HTTPS temporária aponta para seu receptor local.

Se não acontecer: Não reutilize uma URL de túnel encerrado. Este túnel é somente para teste.

Passo 4

DryRun - crie um destino em Webhooks com URL completa: URL_DO_TUNEL/hook/SEGREDO. Use Test, se disponível. Em Training-observe > Notifications, habilite finding notifications, escolha o destino, selecione um limiar adequado e desligue deduplicação neste exercício. Salve.

Esperado: O teste chega ao receptor ou há uma mensagem que permite diagnosticar a entrega.

Se não acontecer: O teste pode ter formato diferente de results; não assuma que ele é um finding real.

Passo 5

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/21
```

Esperado: A última mensagem deve indicar a branch lab/21. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 6

Terminal C - crie a SQLi para gerar uma notificação de PR.

```
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m "Lab: webhook notification"
```

Esperado: O commit fica na branch lab/21.

Passo 7

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 21 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 8

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 9

Terminal C - liste os eventos. Copie o nome de um arquivo real para EVENT_FILE e confira repo_full_name, pr_number e results.

```
ls -lt .evidence/events
read -rp 'Event path (.evidence/events/NAME.json): ' EVENT_FILE
jq . "$EVENT_FILE"
```

Esperado: Você distingue o exemplo training/dryrun-training do evento do seu próprio repo.

Se não acontecer: Sem evento real, confira limiar, configuração, finding e URL. Uma não detecção pode não emitir notificação.

Passo 10

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Passo 11

DryRun - desligue notificações e remova o destino temporário de Training-observe. Só depois use Ctrl+C nos Terminais A e B. Guarde um evento selecionado para o módulo 22.

Esperado: Não existe destino ativo apontando para um túnel encerrado.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Webhooks, páginas 1-5; PR Scanning Configuration, páginas 4-6; Cloudflare Quick Tunnels.

API INTEGRATIONS

22. Jira Integration

Mecanismo e decisões técnicas

A documentação propõe integração por intermediários como Tines ou Zapier, em vez de pressupor um conector nativo completo. O fluxo é receber evento, normalizar campos, filtrar risco, procurar ticket existente, criar/atualizar e registrar o resultado. A automação não deve considerar cada notificação como um novo finding.

Este laboratório implementa a parte central com Python e curl, como alternativa didática ao intermediário. Usa o payload genérico results do módulo 21. Os exemplos de eventos de páginas diferentes podem ter formatos distintos; não misture um objeto finding isolado com um envelope results sem normalização.

Jira Cloud REST v3 usa Atlassian Document Format para description. Projeto e tipo de issue precisam existir e o usuário precisa poder criar aquele tipo. O exemplo usa token pessoal de API sem escopos específicos via domínio do site para um script local. Se sua política exigir token com escopos/gateway api.atlassian.com, adapte a base e escopos conforme a documentação Atlassian da sua conta.

A etiqueta do curso deriva de repo + finding ID. Isso ajuda a pesquisar duplicatas, mas uma sequência “pesquisar e criar” não é atômica: dois workers podem criar duas issues. Uma integração de produção precisa de persistência e controle de concorrência, além de tratamento de limites e falhas parciais.

Caso de uso

Um finding High reaparece após um novo commit. Em vez de abrir outro ticket, o fluxo encontra a issue pela identidade e atualiza a evidência. A prioridade Jira deve seguir um mapeamento aprovado; o laboratório não presume que um nome de prioridade exista em toda instância.

Antes de começar

Módulo 21 ou amostra do kit. Requer Jira Cloud e projeto de teste com permissão Create issues; acesso ao GitHub não fornece acesso Jira.

Laboratório guiado 22

Passo 1

Navegador - abra seu projeto Jira e anote a chave, por exemplo LAB. Crie um token pessoal de API na conta Atlassian. O roteiro usa token sem escopos específicos no domínio do site; se sua organização exigir token com escopos/gateway, peça a base correta ao administrador antes de continuar.

Esperado: Você tem site, email, chave de projeto e token. Nenhum deles é um token DryRun.

Se não acontecer: Sem Jira, para teste exclusivamente local execute export JIRA_PROJECT=LAB e export JIRA_ISSUETYPE_ID=10001, vá ao passo 4 e encerre depois da geração. Esses valores são exemplos; não envie o payload a um site real.

Passo 2

Terminal - digite os valores solicitados. Não inclua uma barra final na URL do site. A leitura do token não mostra caracteres na tela.

```
source "$HOME/dryrun-student/session.sh"
read -rp 'Jira site (https://YOUR-SITE.atlassian.net): ' JIRA_BASE
read -rp 'Atlassian email: ' JIRA_EMAIL
read -rp 'Project key: ' JIRA_PROJECT
read -rsp 'Jira API token: ' JIRA_API_TOKEN; echo
export JIRA_BASE JIRA_EMAIL JIRA_PROJECT JIRA_API_TOKEN
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$JIRA_BASE/rest/api/3/myself" > .evidence/jira-me.json
jq '{accountId,displayName}' .evidence/jira-me.json
```

Esperado: A resposta identifica você.

Se não acontecer: 401: confirme email/token/base. Não prossiga se a consulta de identidade falhar.

Passo 3

Terminal - consulte tipos permitidos e selecione o ID do tipo desejado. Depois veja os campos obrigatórios. Não suponha que todo projeto use Task ou o mesmo ID.

```
META="$JIRA_BASE/rest/api/3/issue/createmeta/$JIRA_PROJECT/issuetypes"
curl --fail-with-body -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$META" > .evidence/jira-types.json
jq . .evidence/jira-types.json
read -rp 'Allowed issue type ID: ' JIRA_ISSUETYPE_ID
export JIRA_ISSUETYPE_ID
curl --fail-with-body -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$META/$JIRA_ISSUETYPE_ID" > .evidence/jira-fields.json
jq . .evidence/jira-fields.json
```

Esperado: Você confirma campos comuns e qualquer exigência adicional.

Se não acontecer: Se houver campo obrigatório extra, ajuste o JSON com orientação do administrador antes do POST.

Passo 4

Terminal - gere primeiro o payload com a amostra sintética. O número 0 seleciona o primeiro finding da lista results. O script não chama o Jira.

```
python3 "$KIT/jira_payload.py" "$KIT/event-example.json" 0
jq . .evidence/jira-create.json
LABEL=$(cat .evidence/jira-label.txt)
```

Esperado: Você vê project, issuetype, summary, labels e description em ADF.

Se não acontecer: Para evento real, substitua o caminho da amostra por EVENT_FILE do módulo 21 e confira se results existe.

Passo 5

Terminal - procure um ticket com a mesma identidade antes de criar. Se a busca falhar, pare; falha não significa lista vazia.

```
curl --fail-with-body --silent --show-error --get \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  --data-urlencode "jql=project = \"$JIRA_PROJECT\" AND labels = \"$LABEL\"" \
  --data-urlencode 'maxResults=20' \
  "$JIRA_BASE/rest/api/3/search/jql" > .evidence/jira-search.json
jq . .evidence/jira-search.json
```

Esperado: Só prossiga ao POST se a busca válida não encontrar a issue. Se encontrar, anote sua key e pule a criação.

Passo 6

Terminal - com payload revisado e busca válida sem correspondência, crie a issue uma única vez. Depois copie sua key.

```
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" -H 'Content-Type: application/json' \
  --data-binary @.evidence/jira-create.json \
  "$JIRA_BASE/rest/api/3/issue" > .evidence/jira-created.json
jq . .evidence/jira-created.json
read -rp 'Issue key (created or existing): ' ISSUE_KEY
```

Esperado: O retorno contém uma key, por exemplo LAB-123. Abra-a no Jira e confira os campos.

Se não acontecer: 400: campos/ADF; 403: permissão. Se a resposta se perder, pesquise antes de repetir POST para evitar duplicata.

Passo 7

Terminal - para encerrar o ticket de teste, consulte transições e selecione o ID da opção de conclusão disponível. IDs variam.

```
TRANS="$JIRA_BASE/rest/api/3/issue/$ISSUE_KEY/transitions"
curl --fail-with-body -u "$JIRA_EMAIL:$JIRA_API_TOKEN" "$TRANS"
read -rp 'Completion transition ID: ' TRANSITION_ID
jq -n --arg id "$TRANSITION_ID" '{transition:{id:$id}}' \
  > .evidence/jira-transition.json
curl --fail-with-body -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  -H 'Content-Type: application/json' \
  --data-binary @.evidence/jira-transition.json "$TRANS"
```

Esperado: Confira o novo status no navegador; a transição pode retornar corpo vazio.

Se não acontecer: Se não existir transição de conclusão, use o fluxo autorizado do projeto ou peça ajuda ao administrador.

Passo 8

Conta Atlassian - revogue o token de laboratório quando não houver consumidor restante. No terminal, descarte a variável.

```
unset JIRA_API_TOKEN
```

Esperado: A issue de teste está identificada/encerrada e o token não ficou em arquivo Git.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Jira Integration, páginas 1-5; Atlassian Basic auth, Issues e Issue search REST v3.

API INTEGRATIONS

23. API Access Keys

Mecanismo e decisões técnicas

Chaves permitem acesso programático sem reutilizar a sessão interativa do dashboard. A criação exige escolher pelo menos um escopo. A chave pode dar acesso a mais de uma conta conforme sua autorização; por isso accounts é o primeiro passo de descoberta. O segredo completo é mostrado uma única vez.

Planeje a chave pelo consumidor: exportador somente leitura, integração de automação ou agente de remediação. Use os menores escopos que permitam a operação real e revise acesso quando o consumidor mudar. Os limites de uso devem ser consultados na conta; o curso não pressupõe um número fixo de requisições por minuto.

Rotação tem uma sequência: criar substituta, atualizar consumidor, verificar operação e revogar antiga. Revogar primeiro pode interromper a integração. Em um incidente de exposição, a prioridade pode ser revogar imediatamente e restaurar o serviço com uma chave nova conforme o processo de resposta.

Caso de uso

O token do receptor de automação foi usado também por uma estação de desenvolvimento. Separar chaves permite revogar a estação sem interromper o serviço e torna o propósito de cada credencial mais claro.

Antes de começar

Preparação Chave DryRun; Admin para criar/revogar chaves. Use somente chaves do curso.

Laboratório guiado 23

Passo 1

DryRun - abra Settings > Access Keys e identifique a chave usada no curso. Anote nome e propósito, nunca o segredo no caderno.

Esperado: Você sabe qual consumidor usa a chave que será substituída.

Se não acontecer: Não revogue uma chave compartilhada cuja origem você não conhece.

Passo 2

DryRun - crie “training-replacement” com os escopos de leitura necessários. Copie o segredo uma vez para um gerenciador seguro. Terminal - teste a substituta sem trocar a chave atual ainda.

```
read -rsp 'Replacement DryRun key: ' DRYRUN_NEW_KEY; echo
curl --fail-with-body --silent --show-error \
  -H "Authorization: Bearer $DRYRUN_NEW_KEY" \
  'https://simple-api.dryrun.security/v1/accounts'
```

Esperado: A chave nova consulta as contas esperadas.

Se não acontecer: Se 403 ocorrer em outras operações, revise o escopo necessário; não conceda todos automaticamente.

Passo 3

Terminal - mantenha a antiga apenas para testar sua revogação, ative a nova e reinicie clientes que herdaram o ambiente antigo.

```
OLD_TRAINING_KEY="$DRYRUN_API_KEY"
export DRYRUN_API_KEY="$DRYRUN_NEW_KEY"
unset DRYRUN_NEW_KEY
dryget accounts
```

Esperado: O consumidor de leitura funciona com a substituta.

Passo 4

DryRun - revogue somente a chave antiga identificada. Terminal - confirme que ela deixa de autorizar e descarte-a.

```
curl --silent --show-error --output .evidence/revoked-response.json \
  --write-out '%{http_code}
' \
  -H "Authorization: Bearer $OLD_TRAINING_KEY" \
  'https://simple-api.dryrun.security/v1/accounts'
unset OLD_TRAINING_KEY
```

Esperado: A antiga é recusada; a nova continua funcionando.

Se não acontecer: Se a antiga continuar autorizando, confirme qual chave foi revogada no painel antes de concluir.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: API Access Keys, páginas 1-2.

API INTEGRATIONS

24. Remediation

Mecanismo e decisões técnicas

Remediação conecta o finding a uma ferramenta de programação por IA para preparar uma alteração contextualizada. A documentação cita Codex, Claude Code, Cursor, GitHub Copilot, Windsurf e VS Code. A integração combina chave, MCP e skill, mas a correção ainda precisa ser revisada e verificada.

O ciclo completo é: identificar finding e SHA, reproduzir a condição, preparar patch mínimo, executar teste que falhava, revisar diff, atualizar a PR e aguardar scan do novo commit. “O agente disse que corrigiu” não é critério de conclusão. Um patch pode remover o sintoma, quebrar compatibilidade ou deixar outro caminho vulnerável.

Em SCA, atualização exige avaliar compatibilidade e regenerar lockfile com o gerenciador do projeto. Em DeepScan, a correção deve partir do estado atual do código; a linha de um scan antigo pode ter mudado. Vincule o ticket e o finding ao novo patch sem assumir que posição de linha é uma identidade estável.

Caso de uso

Um finding indica concatenação SQL. O agente pode corrigir com binding de parâmetros e preservar a resposta da API. O teste de regressão verifica que o payload de injeção passa a ser dado literal e que a busca normal continua funcionando.

Antes de começar

Módulos 3 e 19; chave carregada; Codex autenticado. Este laboratório instala a skill antes de usá-la, sem depender do módulo seguinte.

Laboratório guiado 24

Passo 1

Terminal - instale as skills DryRun e selecione Codex quando solicitado. Escolha o escopo pessoal (user/global), fora do repo de exercícios, para não criar arquivos não versionados antes de abrir a branch.

```
npx skills@latest add DryRunSecurity/external-plugin-marketplace
```

Esperado: O instalador informa onde colocou as skills. Reinicie uma sessão Codex já aberta.

Se não acontecer: Se falhar, registre a mensagem; não suponha que a skill ficou disponível.

Passo 2

Terminal - entre no repositório de exercícios e crie a branch deste módulo. Branch é uma linha de trabalho separada da main. O comando lab_branch é um auxiliar do kit, explicado no apêndice.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/24
```

Esperado: A última mensagem deve indicar a branch lab/24. Só continue depois disso.

Se não acontecer: Se houver arquivos modificados, execute git status e resolva o trabalho anterior. Se a branch já existir, retome-a com git switch e confira a PR; não apague a branch para tentar novamente.

Passo 3

Terminal - prepare a vulnerabilidade e confirme a falha do teste antes da correção.

```
python3 "$KIT/change.py" unsafe
python3 -m unittest -v
git add app.py
git commit -m 'Lab: AI remediation target'
```

Esperado: Falha esperada no teste de injeção.

Passo 4

Terminal e GitHub - publique a branch e abra uma pull request (PR), isto é, um pedido de revisão antes de integrar a alteração. Depois abra a página da PR no navegador.

```
lab_pr "Lab 24 - DryRun training"
gh pr view --web
```

Esperado: O navegador mostra uma PR aberta com base main e a branch do módulo como origem. Não clique em Merge.

Se não acontecer: Se a PR já existir, use gh pr view --web. Se o push falhar, confira gh auth status e git remote -v; não avance esperando um scan de uma alteração que não chegou ao GitHub.

Passo 5

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 6

Terminal - abra o Codex. No chat, informe o repo, número da PR, finding ID e peça o patch mínimo, sem commit, push ou merge automático. Se não houver finding, declare que a evidência é o teste local, não um achado DryRun.

```
codex --sandbox workspace-write
```

Esperado: Peça: “Corrija SQLi em app.py com parâmetros vinculados. Preserve search. Execute os testes. Não altere testes, triagem, configuração nem severidade para esconder a falha.”

Se não acontecer: Se o agente pedir permissão, revise a ação no fluxo do cliente. Não aprove mudanças fora do repo do exercício.

Passo 7

Terminal - após encerrar a sessão, revise o diff e rode os testes por conta própria. Só então salve e envie o patch.

```
git status --short
git diff
python3 -m unittest -v
git add app.py
git commit -m 'Verified SQL injection repair'
git push
```

Esperado: Os dois testes passam e o patch não enfraquece o teste nem altera a política.

Se não acontecer: Se o agente já tiver commitado, confira git log -1 antes de repetir commit. Se testes foram alterados indevidamente, revise antes de publicar.

Passo 8

GitHub e DryRun - na PR, abra Checks e depois consulte o scan correspondente no DryRun. No terminal, guarde a evidência. Atualize a página enquanto estiver em andamento. Se após cerca de 10 minutos não houver atividade, use o diagnóstico; esse tempo é um ponto de investigação do curso, não um SLA.

```
lab_evidence
```

Esperado: Confira o SHA: ele deve ser o mesmo exibido em git rev-parse HEAD. Um check de falha pode ser a detecção esperada. Um comentário ausente não comprova scan limpo.

Se não acontecer: Se não existir check/scan, confirme a instalação da App e o repo selecionado. Se o scan terminou sem finding, registre isso e confira diff/agentes; não declare a vulnerabilidade inexistente.

Passo 9

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: Remediation, páginas 1-2; DryRun Skill, páginas 1-3.

API INTEGRATIONS

25. DryRun Skill

Mecanismo e decisões técnicas

As skills adicionam instruções operacionais ao agente. A documentação descreve três skills: workflows de autoria/revisão de PR e remediação, incluindo dryrun-finding-remediation para DeepScan e SCA fora de PRs abertas. O instalador detecta o cliente e posiciona os arquivos correspondentes. Consulte Settings > Integrations para instruções específicas do seu agente.

No fluxo de autoria, o agente prepara uma branch e abre PR, permitindo o scan antes do merge. No fluxo de revisão, acompanha resultados após cada commit. Na remediação, usa o finding e seu contexto para propor uma correção. A skill de findings exige DRYRUN_API_KEY e permite consultar achados já presentes na base principal.

Há um detalhe operacional importante: o fluxo documentado fala em aguardar comentários, mas a configuração padrão pode não comentar scans limpos. O workflow deve conferir conclusão dos checks/scan e ausência de findings, sem esperar indefinidamente por um comentário que não será publicado. Uma branch com push direto para main não fornece o ciclo de PR desejado.

Caso de uso

Você pede ao agente uma pequena melhoria. A skill orienta a criação da PR, apresentação dos findings e uma nova checagem após correções. O ganho é incorporar a segurança ao processo de entrega; a skill não substitui permissões do GitHub nem enforcement de branch.

Antes de começar

Módulos 19 e 24; skills instaladas. Use uma branch nova; não reutilize a PR encerrada do módulo 24.

Laboratório guiado 25

Passo 1

Terminal - carregue a sessão e confira árvore limpa. Se ainda não instalou as skills, execute o comando indicado; se já instalou, não precisa repetir.

```
source "$HOME/dryrun-student/session.sh"
git switch main
git status --short
# Only if not already installed:
# npx skills@latest add DryRunSecurity/external-plugin-marketplace
codex --sandbox workspace-write
```

Esperado: O cliente foi iniciado no repo de exercícios com as skills disponíveis.

Se não acontecer: Se o cliente não reconhecer a skill, confirme local de instalação e reinicie a sessão.

Passo 2

Chat do Codex - peça: “Use as skills DryRun instaladas. Na branch lab/25, documente no README por que parâmetros SQL são usados. Abra uma PR para main, confira o scan do SHA final e apresente resultados. Não faça merge.”

Esperado: O agente cria uma PR e explica o resultado ou a limitação de acesso.

Se não acontecer: Se não houver comentário, solicite verificação dos checks/scan; não deixe o agente esperando para sempre por comentário de scan limpo.

Passo 3

Terminal - após sair do Codex, confira a PR e compare o SHA com os checks.

```
gh pr view --json number,url,headRefOid,state
lab_evidence
git log -1 --oneline
```

Esperado: A PR é do seu repo, o diff é documental e o resultado se refere ao último commit.

Se não acontecer: Se a PR não existir, examine git status e a explicação do agente antes de executar comandos adicionais.

Passo 4

Opcional, chat - peça a dryrun-finding-remediation para listar findings DeepScan do repo com IDs, sem editar. Escolha apenas findings reais para correção futura.

Esperado: Zero findings é uma resposta válida; não introduza uma falha em main para fabricar um caso.

Passo 5

Terminal - encerre a PR de exercício sem mesclar. Isso mantém a main segura para o próximo módulo.

```
lab_close
git branch --show-current
git status --short
```

Esperado: A branch atual é main e git status --short não mostra alterações. As evidências continuam em .evidence.

Se não acontecer: Se a PR já foi fechada, use git switch main apenas depois de verificar que não há alterações locais. Não use reset --hard.

Passo 6

Terminal e DryRun - ao terminar todo o curso, remova apenas a conexão MCP de treinamento, revogue chaves sem consumidores restantes e desligue integrações temporárias.

```
codex mcp remove dryrun-training
unset DRYRUN_API_KEY JIRA_API_TOKEN WEBHOOK_PATH_KEY
```

Esperado: As credenciais não ficaram no repo e os serviços temporários foram encerrados.

Se não acontecer: Não remova conexões pessoais ou chaves de outros consumidores.

Checkpoint do aluno

No caderno, registre o que fez, a evidência observada e qualquer etapa não executada. Explique o resultado com suas palavras. Só avance quando entender as diferenças entre o resultado esperado e o observado. A limpeza está nos últimos passos aplicáveis deste laboratório.

Fonte: DryRun Skill, páginas 1-3; Remediation, páginas 1-2.

Exercício final - demonstre o ciclo completo

1. Prepare uma nova ocorrência

Depois de concluir os módulos, carregue a sessão e crie uma branch nova. Introduza a SQLi, rode o teste e confirme que a falha é a mesma estudada no módulo 3. Salve e abra PR.

```
source "$HOME/dryrun-student/session.sh"
lab_branch lab/final
python3 "$KIT/change.py" unsafe
python3 -m unittest -v
git add app.py
git commit -m 'Final exercise: vulnerable query'
lab_pr 'Final exercise - DryRun lifecycle'
lab_evidence
```

2. Investigue antes de corrigir

Na PR e no DryRun, identifique repo, SHA, scan e finding, se houver. Explique origem da entrada, operação SQL e controle ausente. Registre In Progress somente se houver um finding real. Se o scanner não detectar, documente a diferença em relação à prova local.

3. Corrija e demonstre

Use o Codex conforme o módulo 24 ou aplique a correção determinística abaixo. Não altere o teste para fazê-lo passar. Depois envie o commit e confira o scan final.

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Final exercise: verified fix'
git push
lab_evidence
```

4. Entregue sua evidência

Sua entrega contém: repo/PR/SHA, resultado dos testes antes/depois, explicação da falha, resultado do scan final e limitações. Se executou Jira, inclua a key do ticket de teste. Se não possuía Jira ou Codex, declare a etapa não executada. Isso é melhor que apresentar uma integração simulada como real.

Critério de conclusão: você consegue repetir a prática e explicar por que a correção funciona, sem confundir teste local com evidência remota. Feche a PR sem merge e faça a limpeza indicada no módulo 25.

```
lab_close
gh pr list --state open --limit 30
git status --short
```

Diagnóstico rápido e retomada

Comando não encontrado

Confira a ferramenta com command -v NOME. Para lab_branch, lab_pr, lab_evidence, lab_close ou dryget, carregue session.sh. Para gh/codex/jq/python3, volte à instalação. Uma sessão nova pode não ter o PATH configurado pelo instalador; siga seus Next steps.

Git e branches

Use pwd, git status e git branch --show-current antes de repetir comandos. Se a branch já existir, retome-a. Se uma PR já existir, abra-a com gh pr view --web. Não crie outra PR só porque o comentário DryRun ainda não chegou. Se houver alterações locais, entenda e salve o trabalho antes de trocar de branch.

Scan não aparece ou não comenta

Confirme App/repo, organização, PR e SHA. Confira draft/ready, comentários, Silent mode e término do scan. Comentário ausente é compatível com scan limpo, mas também com falta de execução; a evidência decisiva vem do check/painel. Em execução com finding, registre severidade e configuração antes de explicar blocking.

Credenciais e HTTP

401 pede revisão da credencial. 403 pede revisão de autorização/escopo. 404 pode indicar ID incorreto ou recurso oculto. 429 pede respeitar limite/Retry-After. Para 5xx, salve horário, endpoint e request_id quando disponível. Nunca envie o token junto ao pedido de suporte.

Quando pedir ajuda

Informe módulo e passo, comando sem segredo, pasta atual, resultado esperado, mensagem real e IDs relevantes. Diga se o erro é local, GitHub, DryRun, Codex ou Jira. Não envie captura que exponha tokens. Essa informação permite ao instrutor ajudar sem refazer toda a preparação.

Apêndice A - código completo do kit

Estes arquivos são auxiliares didáticos, não comandos oficiais DryRun. O kit usa mensagens em inglês para funcionar da mesma forma nos dois idiomas; o roteiro explica os resultados. Copie cada bloco para o arquivo indicado, em material/laboratorios, caso não use o clone/ZIP. Não há necessidade de transcrever se você já baixou os arquivos. Apenas app.py e test_app.py vão para o repo inicial de exercícios.

lab.sh

```
# Course helpers. Source this file from the generated session.sh.
export DRYRUN_BASE='https://simple-api.dryrun.security/v1'
lab_branch() {
  test -n "$GH_REPO" || { echo 'Load session.sh first.'; return 1; }
  test -z "$(git status --porcelain)" || {
    echo 'STOP: local changes exist. Run git status and finish them.'; return 1;
  }
  git switch main && git pull --ff-only && git switch -c "$1"
}
lab_pr() {
  test "$(git branch --show-current)" != main || {
    echo 'STOP: create an exercise branch first.'; return 1;
  }
  git push -u origin HEAD || return
  gh pr create --base main --title "$1" --body \
    'DryRun student exercise. Do not merge vulnerable code.'
}
lab_evidence() {
  local n sha
  n=$(gh pr view --json number --jq .number) || return
  sha=$(gh pr view --json headRefOid --jq .headRefOid) || return
  mkdir -p .evidence
  printf 'PR %s; remote SHA %s\n' "$n" "$sha"
  git rev-parse HEAD
  gh api "repos/$GH_REPO/commits/$sha/check-runs" \
    > ".evidence/pr-$n-checks.json" || return
  gh api --paginate "repos/$GH_REPO/issues/$n/comments" \
    > ".evidence/pr-$n-comments.json" || return
  gh pr checks
  echo 'Read the check results above; failure can be an expected finding.'
}
lab_close() {
  test -z "$(git status --porcelain)" || {
    echo 'STOP: local changes exist. Finish/review them first.'; return 1;
  }
  gh pr close || return
  git switch main
}
dryget() {
  test -n "$DRYRUN_API_KEY" || {
    echo 'STOP: load DRYRUN_API_KEY in this terminal.' >&2; return 1;
  }
  curl --fail-with-body --silent --show-error \
    -H "Authorization: Bearer $DRYRUN_API_KEY" "$DRYRUN_BASE/$1"
}
```

save_session.py

```
"""Save non-secret student paths; no token is stored."""
from pathlib import Path
import re
import shlex
import sys


def save(repo, root):
    if not re.fullmatch(r'[A-Za-z0-9_.-]+/[A-Za-z0-9_.-]+', repo):
        raise ValueError('Expected OWNER/REPO, not a URL')
    root = Path(root).resolve()
    kit = root / 'material' / 'laboratorios'
    work = root / 'repo'
    if not (kit / 'lab.sh').is_file() or not (work / '.git').exists():
        raise ValueError('Expected material/laboratorios and initialized repo')
    q = shlex.quote
    content = '\n'.join([
        '# Generated course session: no credentials.',
        'export KIT=' + q(str(kit)),
        'export GH_REPO=' + q(repo),
        'cd ' + q(str(work)) + ' || return',
        'mkdir -p .evidence',
        'source "$KIT/lab.sh"',
        '',
    ])
    (root / 'session.sh').write_text(content)
    print('Session saved:', root / 'session.sh')


if __name__ == '__main__':
    if len(sys.argv) != 2:
        raise SystemExit('Usage: python3 save_session.py OWNER/REPO')
    save(sys.argv[1], Path.home() / 'dryrun-student')
```

app.py

```
"""Teaching application: loopback only; no real authentication."""
import json
import sqlite3
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import parse_qs, urlparse


def database():
    db = sqlite3.connect(':memory:')
    db.execute('CREATE TABLE products (name TEXT, price INTEGER)')
    db.executemany('INSERT INTO products VALUES (?, ?)',
                   [('book', 10), ('pencil', 2)])
    return db


def search(db, term):
    return db.execute(
        'SELECT name, price FROM products WHERE name = ?',
        (term,)).fetchall()


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        parsed = urlparse(self.path)
        if parsed.path != '/search':
            self.send_error(404)
            return
        term = parse_qs(parsed.query).get('q', [''])[0]
        with database() as db:
            result = search(db, term)
        body = json.dumps(result).encode()
        self.send_response(200)
        self.send_header('Content-Type', 'application/json')
        self.end_headers()
        self.wfile.write(body)


if __name__ == '__main__':
    HTTPServer(('127.0.0.1', 8090), Handler).serve_forever()
```

test_app.py

```
import unittest
from app import database, search


class SearchTest(unittest.TestCase):
    def test_known_product(self):
        with database() as db:
            self.assertEqual(search(db, 'book'), [('book', 10)])

    def test_injection_is_literal(self):
        with database() as db:
            self.assertEqual(search(db, "' OR 1=1 --"), [])


if __name__ == '__main__':
    unittest.main()
```

change.py

```
"""Changes only the training app.py after checking its current state."""
import pathlib
import sys
p = pathlib.Path('app.py')
safe = """return db.execute(
        'SELECT name, price FROM products WHERE name = ?',
        (term,)).fetchall()"""
unsafe = """query = "SELECT name, price FROM products WHERE name = '" + term + "'"
    return db.execute(query).fetchall()"""
mode = sys.argv[1]
assert mode in ('unsafe', 'safe')
before, after = (safe, unsafe) if mode == 'unsafe' else (unsafe, safe)
text = p.read_text()
if text.count(before) != 1:
    raise SystemExit('Unexpected app.py state; inspect the diff.')
p.write_text(text.replace(before, after))
```

receiver.py

```
"""Teaching receiver. Saves JSON; does not forward to Jira."""
import hashlib
import hmac
import json
import os
from pathlib import Path
from http.server import BaseHTTPRequestHandler, HTTPServer

KEY = os.environ['WEBHOOK_PATH_KEY']
OUT = Path('.evidence/events')
OUT.mkdir(parents=True, exist_ok=True)


class Handler(BaseHTTPRequestHandler):
    def log_message(self, *args):
        pass  # Do not log the URL containing the training path secret.

    def do_POST(self):
        if not hmac.compare_digest(self.path, '/hook/' + KEY):
            self.send_error(404)
            return
        try:
            size = int(self.headers.get('Content-Length', '0'))
            if not 0 < size <= 1024 * 1024:
                self.send_error(413)
                return
            data = json.loads(self.rfile.read(size))
            if not isinstance(data, dict):
                raise ValueError('JSON object expected')
        except (ValueError, json.JSONDecodeError):
            self.send_error(400)
            return
        raw = json.dumps(data, sort_keys=True).encode()
        digest = hashlib.sha256(raw).hexdigest()
        dest = OUT / (digest + '.json')
        try:
            with dest.open('xb') as f:
                f.write(raw)
            print('Event saved:', dest, flush=True)
        except FileExistsError:
            print('Duplicate ignored:', digest, flush=True)
        self.send_response(202)
        self.end_headers()
        self.wfile.write(b'{"accepted":true}')


if __name__ == '__main__':
    print('Listening on 127.0.0.1:8080. Stop with Ctrl+C.', flush=True)
    HTTPServer(('127.0.0.1', 8080), Handler).serve_forever()
```

jira_payload.py

```
"""Converts ONE generic webhook finding to a Jira Cloud v3 payload."""
import json
import os
import sys
from pathlib import Path

event = json.loads(Path(sys.argv[1]).read_text())
index = int(sys.argv[2]) if len(sys.argv) > 2 else 0
finding = event['results'][index]
repo = event.get('repo_full_name', 'repository-not-provided')
ident = str(finding['id'])
import hashlib
label = 'dryrun-' + hashlib.sha256(
    (repo + ':' + ident).encode()).hexdigest()[:24]
text = '\n'.join([
    'DryRun finding: ' + ident,
    'Repository: ' + repo,
    'PR: ' + str(event.get('pr_number', '')),
    'Risk: ' + str(finding.get('risk', '')),
    'File: ' + str(finding.get('filename', '')),
    'Line: ' + str(finding.get('line_start', '')),
    str(finding.get('description', '')),
    'Dashboard: ' + str(event.get('dashboard_url', '')),
])
payload = {'fields': {
    'project': {'key': os.environ['JIRA_PROJECT']},
    'issuetype': {'id': os.environ['JIRA_ISSUETYPE_ID']},
    'summary': ('[DryRun] ' + str(finding.get('type', 'Finding'))
                + ' - ' + repo)[:240],
    'labels': [label],
    'description': {'type': 'doc', 'version': 1, 'content': [
        {'type': 'paragraph', 'content': [{'type': 'text', 'text': text}]}]},
}}
Path('.evidence/jira-create.json').write_text(json.dumps(payload, indent=2))
Path('.evidence/jira-label.txt').write_text(label)
print('Payload ready. Label:', label)
```

event-example.json

```
{
  "repo_full_name": "training/dryrun-training",
  "pr_number": 1,
  "dashboard_url": "https://app.dryrun.security",
  "results": [{
    "id": "lab-sqli-001", "type": "SQL Injection",
    "filename": "app.py", "line_start": 20, "line_end": 21,
    "description": "Synthetic example: input concatenated into a query.",
    "risk": "High", "finding_type": "pullrequest"
  }]
}
```

Apêndice B - fontes e limites

Base: 25 PDFs oficiais fornecidos pelo participante, datados de 20/10/2026. As páginas citadas nos módulos correspondem a esses PDFs. Revisão pedagógica e verificação adicional de comandos: 23/10/2026. O guia anterior é substituído por esta edição orientada ao aluno.

API Access Keys - DryRun Security Docs.pdf - https://docs.dryrun.security/api-access-keys

Code Security Intelligence - DryRun Security Docs.pdf - https://docs.dryrun.security/code-security-intelligence

Custom Code Policies - DryRun Security Docs.pdf - https://docs.dryrun.security/custom-code-policies

Documentation - DryRun Security Docs.pdf - https://docs.dryrun.security/

DryRun API - DryRun Security Docs.pdf - https://docs.dryrun.security/dryrun-api

DryRun Skill - DryRun Security Docs.pdf - https://docs.dryrun.security/dryrun-skill

Finding Tuning with Feedback - DryRun Security Docs.pdf - https://docs.dryrun.security/finding-tuning

IaC Scanning - DryRun Security Docs.pdf - https://docs.dryrun.security/iac-scanning

Jira Integration - DryRun Security Docs.pdf - https://docs.dryrun.security/jira-integration

Language and Framework Support - DryRun Security Docs.pdf - https://docs.dryrun.security/language-support

MCP - DryRun Security Docs.pdf - https://docs.dryrun.security/mcp

PR Blocking - DryRun Security Docs.pdf - https://docs.dryrun.security/pr-blocking

PR Scanning - DryRun Security Docs.pdf - https://docs.dryrun.security/pr-scanning

PR Scanning Configuration - DryRun Security Docs.pdf - https://docs.dryrun.security/pr-scanning-configuration

Permissions - DryRun Security Docs.pdf - https://docs.dryrun.security/permissions

Quick Start - DryRun Security Docs.pdf - https://docs.dryrun.security/quick-start

Remediation - DryRun Security Docs.pdf - https://docs.dryrun.security/remediation

Reporting - DryRun Security Docs.pdf - https://docs.dryrun.security/reporting

Repository Context - DryRun Security Docs.pdf - https://docs.dryrun.security/repository-context

Repository Scanning with DeepScan - DryRun Security Docs.pdf - https://docs.dryrun.security/deepscan

Risk Register - DryRun Security Docs.pdf - https://docs.dryrun.security/risk-register

SCA - DryRun Security Docs.pdf - https://docs.dryrun.security/sca

Secrets Scanning - DryRun Security Docs.pdf - https://docs.dryrun.security/secrets-scanning

Vulnerability Coverage Matrix - DryRun Security Docs.pdf - https://docs.dryrun.security/vulnerability-coverage-matrix

Webhooks - DryRun Security Docs.pdf - https://docs.dryrun.security/webhooks

Referências externas

GitHub CLI - https://cli.github.com/manual/

GitHub rulesets - https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/creating-rulesets-for-a-repository

Codex CLI - https://learn.chatgpt.com/docs/codex/cli

Codex commands - https://learn.chatgpt.com/docs/cli/reference

Codex MCP - https://learn.chatgpt.com/docs/extend/mcp

Jira authentication - https://developer.atlassian.com/cloud/jira/platform/basic-auth-for-rest-apis/

Jira issues, metadata, transitions - https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-issues/

Jira JQL search - https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-issue-search/

Homebrew - https://brew.sh

Cloudflare Quick Tunnels - https://developers.cloudflare.com/tunnel/get-started/quick-tunnels/

DryRun OpenAPI - https://simple-api.dryrun.security/api-docs/v1/swagger.yaml

O que foi validado na autoria

A sintaxe dos blocos Bash e dos scripts Python foi verificada. A base segura passa nos testes; a versão vulnerável falha no teste de injeção; a correção volta a passar. O gerador de sessão e a transformação de payload Jira foram testados localmente. A recepção de webhook foi exercitada com dados sintéticos. As integrações reais exigem suas contas, permissões e configuração; scans DryRun, criação de issues Jira e execução Codex nas suas contas não foram realizados como parte dessa validação. A publicação dos materiais no GitHub é uma operação distinta dos laboratórios.

Interfaces, planos e nomes de opções podem mudar. Os passos descrevem controles documentados e informam quando um administrador é necessário. Um recurso não disponível não deve ser substituído por uma chamada de API inventada.